PLAYWRIGHT
INTERVIEW MASTER GUIDE

Every commonly-asked question, explained in plain language - with memory hooks and ready-to-speak interview answers

Prepared for: jashu5446
29 September 2026

INDEX

PLAYWRIGHT INTERVIEW MASTER GUIDE (Fully Explained Edition) 6

SECTION 1 — WHAT IS PLAYWRIGHT? (the first question, always) 7

1.1 Understand it 7

1.2 Supported languages 7

SECTION 2 — PLAYWRIGHT vs SELENIUM vs CYPRESS (guaranteed question) 8

2.1 Understand it — the honest comparison 8

2.2 When is Selenium still the better answer? 8

SECTION 2.5 — THE PLAYWRIGHT COMPONENT FAMILY (asked as "what are the components of Playwright?") 9

SECTION 3 — "EXPLAIN YOUR PLAYWRIGHT FRAMEWORK" (the #1 question) 10

3.1 Understand the standard project 10

3.2 The 60-second script (memorize word-for-word) 10

3.3 MEMORY HOOK for the framework blocks 10

3.4 Why POM in Playwright? 10

SECTION 4 — BROWSER vs CONTEXT vs PAGE (the architecture question) 12

Q: Playwright architecture — Playwright Client, Playwright Server, BrowserDriver, Browser Binaries (know all four by name!) 12

Q: Why do we run npx playwright install after npm install? (the binaries question) 13

4.1 Understand the nesting dolls (Browser -> Context -> Page) 13

4.2 Manual version (you'll be asked to write it) 13

SECTION 5 — AUTO-WAITING (the #1 differentiator — know it cold) 14

5.1 Understand it 14

5.2 Navigational auto-waits 14

5.3 When you DO need explicit waits 14

SECTION 6 — LOCATORS (the modern way — forget XPath-first thinking) 15

6.1 Understand the philosophy 15

6.2 STRICT MODE (a favorite trick question) 15

6.3 Filtering and chaining locators (asked as "how do you handle a row in a table?") 15

6.4 Shadow DOM (the hidden world inside a component) 15

SECTION 7 — WEB-FIRST ASSERTIONS (the expect() system) 18

7.1 Understand it 18

7.2 expect.soft — Playwright's SoftAssert . . . 18
7.3 Negative assertions . . . 18

SECTION 8 — FIXTURES (the plumbing question — "where does page come from?") . . . 19

8.1 Understand it . . . 19
8.2 Custom fixtures — the senior answer (test.extend) . . . 19
8.3 Overriding built-in fixtures . . . 20

SECTION 9 — HOOKS, TAGS & TEST CONTROL . . . 21

9.1 The hooks (vs TestNG's before/after) . . . 21
9.2 Tags, only, skip, fixme, and grep . . . 21
9.3 Serial mode (and why it's a last resort) . . . 21

SECTION 10 — playwright.config.ts (the brain — "explain your config") . . . 22

10.1 A real config, line by line . . . 22
10.2 The concepts each line carries (be able to expand any of them) . . . 22

SECTION 11 — THE END-TO-END FLOW: CONFIG -> HTML REPORT (including test data) . . . 23

11.1 Understand the full journey of one run . . . 23
11.2 Where TEST DATA enters (3 doors — same pattern as Selenium) . . . 23

SECTION 12 — TEST ISOLATION & AUTH (storageState — asked in every serious interview) . . . 25

12.1 Understand the problem . . . 25
12.2 The fix: login ONCE, save the session, reuse it . . . 25

SECTION 13 — TABS, DIALOGS & IFRAMES (scenario corner) . . . 26

13.1 Multiple tabs/popups . . . 26
Q: 4 tabs open — how do you come back to the ORIGINAL tab? (bringToFront + context.pages — multi-tab in depth) . . . 27
Q: 5 tabs are open and I am on the 5th tab — how do I go to the 2nd or 3rd tab? . . . 27
13.2 JavaScript dialogs (alert/confirm) . . . 28
13.3 Iframes — frameLocator (the easiest of any tool) . . . 28

SECTION 14 — NETWORK: MOCKING, INTERCEPTING, API TESTING . . . 29

14.1 Understand it . . . 29
14.2 Intercept & mock with page.route (the #1 skill) . . . 29
14.3 API testing with the request fixture . . . 29
14.4 Waiting for network . . . 29

SECTION 15 — EMULATION & VISUAL TESTING (a Playwright specialty) . . . 30

SECTION 16 — FILES, UPLOADS, DOWNLOADS, INPUTS (daily scenario bag) . . . 31

SECTION 17 — DEBUGGING TOOLKIT (asked as "how do you debug failures?") 32

SECTION 18 — PARALLELISM, RETRIES & FLAKINESS 33

SECTION 19 — REPORTING & CI 34

SECTION 20 — DATA-DRIVEN TESTING (Playwright style) 35

SECTION 21 — COMMANDS & VERSIONS (know these cold) 36

21.1 The package.json — required packages with versions (the interview table) 36

21.2 Winston — the logging library (the Log4j of the Node/Playwright world) 36

SECTION 22 — SCENARIO QUESTIONS CORNER (every scenario, fully explained) 38

Q: The element is found but the click fails. How do you handle it? 38

Q: How do you test SPA navigation (React/Angular pages that never reload)? 38

Q: The app generates dynamic IDs. How do you locate elements then? 38

Q: Shadow DOM — how do you handle it? (summary — full deep-dive is Section 6.4) 39

Q: How do you read and verify a specific table row? 39

Q: How do you automate a date picker? 39

Q: Canvas apps and CAPTCHA — the honest answer? 40

Q: Your suite is slow and flaky. How do you triage it? 40

Q: CI is expensive. How do you run fewer browsers without losing coverage? 40

SECTION 23 — CODING QUESTIONS THEY MAKE YOU WRITE LIVE (full code for all ten) 41

23.1 Login test with POM + a custom fixture (the #1 hands-on — own it cold) 41

23.2 Find a table row by name -> delete it -> assert it disappears 41

23.3 Mock an API to return 500 and assert the error banner 42

23.4 Handle a popup with the Promise.all pattern 42

23.5 Reach into an iframe with frameLocator 42

23.6 storageState — log in ONCE for the whole suite (the setup project) 42

23.7 Data-driven login from JSON — a loop with dynamic test titles 43

23.8 Upload a file and verify the success toast 43

23.9 expect.soft — collect ALL form validation messages 43

23.10 Visual screenshot of the navbar (image as assertion) 43

SECTION 24 — RAPID-FIRE, FULLY EXPLAINED (final revision — every line now with its why) . . 44

SECTION 25 — CLOSING ANSWER + SELENIUM-TO-PLAYWRIGHT TRANSLATION 46

SECTION 26 — THE REMAINING CORNER (extras interviewers check) 47

26.1 globalSetup / globalTeardown ("how do you do something ONCE before ALL tests?") 47

26.2 test.step(), test.info() and test.attach() (report polish questions) . . . 47
26.3 expect.poll() and toPass() (advanced assertion questions) . . . 47
26.4 addInitScript — run JS BEFORE the page's own scripts . . . 47
26.5 Component testing (know the concept) . . . 47
26.6 Touch actions (mobile emulation) . . . 48
26.7 Playwright in Java / Python / .NET . . . 48
26.8 BDD with Playwright . . . 48

SECTION 27 — ASSIGNMENTS (the live-round coding bank — Q15 to Q34) . . . 49

PART A — CODING ASSIGNMENTS . . . 49
PART B — DEBUGGING EXERCISES . . . 54
PART C — INTERVIEW FOLLOW-UPS (the meta questions) . . . 56

SECTION 28 — APPENDIX: THE FINAL REFERENCE PAGES (config file, framework design, libraries, commands) . . . 59

28.1 The COMPLETE playwright.config.ts — everything we covered, in ONE file . . . 59
28.2 Our framework in detail — how we designed it, and how we use it . . . 60
28.3 The library stack — "if we use Excel, which library do we use?" . . . 61
28.4 Commands — from ZERO to running (memorize these cold) . . . 62

PLAYWRIGHT INTERVIEW MASTER GUIDE (Fully Explained Edition)

Built in the same style as the Selenium guide. Every topic has 4 parts:

1. **Understand it** — simple explanation with a real-life analogy, so the concept actually makes sense.
2. **Code** — what it looks like when written.
3. **MEMORY HOOK** — a short phrase to lock it in your brain.
4. **SAY THIS** — the interview-ready spoken answer (20–40 seconds).

SECTION 1 — WHAT IS PLAYWRIGHT? (the first question, always)

1.1 Understand it

Playwright is Microsoft's open-source **end-to-end automation framework** for web apps. Three sentences to truly understand it:

1. **It controls the browser at a level deeper than clicks.** Playwright talks to browsers (Chromium, Firefox, WebKit) over a **WebSocket connection** — one always-open pipe — instead of Selenium's request-response HTTP calls. That makes it faster and lets it listen to browser events (a download started, a popup opened, a network call finished) instead of repeatedly asking "are you done yet?".
2. **It auto-waits.** In Selenium you write explicit waits everywhere. Playwright's actions **automatically wait** for the element to be visible, enabled, and stable before clicking — waits are built into every action. This kills most of the flakiness that made Selenium interviews painful.
3. **It comes as a complete package.** Not just automation: it ships its own **test runner** (`@playwright/test`), its own **assertions** (web-first expect), **reporters**, **trace viewer**, **code generator**, and **API testing** — the whole toolkit in one `npm install`.

The restaurant analogy: Selenium gives you a skilled driver and you arrange the kitchen, menu and bills yourself (TestNG + Maven + Extent + Excel readers). Playwright gives you a **fully staffed restaurant** — driver, examiner, camera crew and accountant in one team.

1.2 Supported languages

Officially: **TypeScript/JavaScript** (primary — everything newest arrives here first), **Java**, **Python**, and **.NET/C#**. Say: "I use TypeScript — it's the most commonly used in the market and gets new features first."

MEMORY HOOK: *"One WebSocket pipe, one npm install, one team."*

SECTION 2 — PLAYWRIGHT vs SELENIUM vs CYPRESS (guaranteed question)

2.1 Understand it — the honest comparison

Point	Playwright	Selenium	Cypress
Backed by	Microsoft	Community	Cypress.io (commercial)
Languages	TS/JS, Java, Python, C#	Java, Python, C#, etc.	JS/TS only
Browsers	Chromium, **Firefox, WebKit (Safari)**	All, incl. real Safari & IE legacy	Chromium-family + Firefox (WebKit experimental)
Communication	**WebSocket** (persistent, 2-way)	HTTP request–response	Runs INSIDE the browser
Auto-wait	**Built into every action**	Manual (implicit/explicit/fluent)	Built-in
Test runner	Built-in (`@playwright/test`)	Needs TestNG/JUnit + Maven	Built-in (Mocha)
Multiple tabs	Yes, natively	Yes, via window handles	Historically limited
iFrames	`frameLocator()` — easy	switchTo().frame	Chained, clunkier
Speed & stability	Fast, low flake	Slower, more flake	Fast
Ecosystem maturity	Younger but growing fastest	Oldest, biggest community, most jobs	Popular in JS-only shops
Parallelism	Built-in workers + sharding	Grid/TestNG setup	Paid (dashboard) for serious parallel

The mature answer when asked "why did your team move to / choose Playwright?":

> *"Three reasons: auto-waiting removed most of our flaky-wait code; one tool gave us the runner, assertions, tracing and reporting instead of gluing Selenium + TestNG + Extent together; and multi-tab, iframe and network mocking are first-class. Selenium is still great and has the biggest community — but for a greenfield TypeScript project, Playwright is more productive."*

MEMORY HOOK: *"Selenium = assembled, Playwright = all-in-one, Cypress = JS-only."*

2.2 When is Selenium still the better answer?

Legacy Java shops, teams with deep existing Selenium frameworks, when you need real Safari/legacy browsers, or when hiring for Java skill is the priority. Saying this shows balance.

SECTION 2.5 — THE PLAYWRIGHT COMPONENT FAMILY (asked as "what are the components of Playwright?")

Understand it: Selenium is a SUITE of separate tools (IDE, RC, WebDriver, Grid). Playwright ships as a FAMILY of components too — but they come in one npm package and work as one team. Interviewers who know Selenium ask exactly this, so learn each by name:

#	Component	What it is	Selenium-world equivalent
1	**Playwright Library** (`npm i playwright`)	The core automation ENGINE — browser control API without any test runner. Used standalone for scraping/scripts.	**Selenium WebDriver**
2	**@playwright/test** (`npm i -D @playwright/test`)	The TEST RUNNER — test(), describe(), expect(), fixtures, config, reporters, retries, parallelism.	**TestNG/JUnit + Maven Surefire**
3	**Playwright Codegen**	Records your clicks and TYPES OUT clean best-practice code (getByRole locators, auto-waits).	**Selenium IDE**
4	**Playwright Inspector**	The interactive DEBUGGER — step through a test, watch each locator resolve live.	(no Selenium equivalent)
5	**Trace Viewer**	Post-run FLIGHT RECORDER — step-by-step DOM snapshots, network, console, timing of a finished run.	(closest: Extent + video, but far weaker)
6	**UI Mode** (`npx playwright test --ui`)	Interactive test explorer — watch mode, time-travel, filter by project/grep, run one test.	(no equivalent)
7	**HTML Reporter**	The built-in report with filters, traces embedded, failure screenshots.	**Extent Reports**
8	**Bundled browsers**	One command downloads Playwright's own Chromium, Firefox & WebKit builds — no separate driver downloads.	**chromedriver/geckodriver + WebDriverManager**
9	**VS Code extension + CLI + MCP**	Run/debug in the editor; the CLI runs suites; MCP lets AI agents drive the browser.	(no equivalent)
—	*(Selenium RC has NO Playwright equivalent — that JavaScript-injection era simply never existed here)*		**Selenium RC**

Two details interviewers love:

- **Library vs Test runner (a real trap):** `playwright` (library) = automation only; `@playwright/test` = library + runner + assertions + reporter. Real projects install `@playwright/test`.
- **Grid equivalent?** Playwright has NO separate Grid — parallelism is **built into the runner** (workers) and **sharding** across CI machines. One tool instead of a Grid farm.

SAY THIS: "Playwright comes as a family: the Library is the automation engine, @playwright/test is the runner, Codegen is the modern Selenium IDE, the Inspector is the debugger, the Trace Viewer is the flight recorder, and UI Mode is the dev experience. And unlike Selenium, there's no separate Grid — workers and sharding are built in."

MEMORY HOOK: *"Library drives, Test judges, Codegen records, Inspector debugs, Trace proves, UI Mode explores."*

SECTION 3 — "EXPLAIN YOUR PLAYWRIGHT FRAMEWORK" (the #1 question)

3.1 Understand the standard project

`npm init playwright@latest` scaffolds everything for you. What a professional project looks like:

```
playwright-framework/
|-- package.json             <- deps: @playwright/test, plus type definitions
|-- playwright.config.ts     <- the BRAIN: browsers, retries, workers, reporters
|-- tsconfig.json            <- TypeScript config
|
+-- src/  (or tests/)        <- the TESTS
|     |-- pages/             <- Page Object classes (LoginPage.ts, CartPage.ts)
|     |-- tests/             <- spec files: login.spec.ts, cart.spec.ts
|     |-- fixtures/          <- custom fixtures (authenticated page, test data)
|     +-- utils/             |-- helpers: api client, data generators
|
|-- test-data/               <- JSON/CSV files + .auth/ storageState files
|-- playwright-report/       <- generated HTML report (gitignored)
+-- test-results/            <- traces, videos, screenshots of the last run
```

3.2 The 60-second script (memorize word-for-word)

> *"Our framework is a **TypeScript Playwright project** using `@playwright/test`. It's structured with **Page Objects** — every page is a class with locators and actions, so a locator change is fixed in one file. The **playwright.config.ts** is the brain: it defines **projects** for Chrome, Firefox and WebKit, sets `baseURL`, **retries** for flaky tests, the number of parallel **workers**, and turns on **traces, videos and screenshots on failure**. Tests use the built-in **fixtures** — `test` and `page` — so every test gets a fresh, isolated browser context automatically. We add **custom fixtures** for things like an already-logged-in page, which reuses a saved **storageState** so we login once, not in every test. Test data comes from **JSON files and fixtures**, and for API calls we use Playwright's built-in **request** client. Assertions are **web-first** — `expect(locator).toBeVisible()` retries until the condition is true. The run produces an **HTML report** with a **trace viewer** — on failure I can open the trace and see a step-by-step video, DOM snapshots and network calls of exactly what happened. The whole thing runs in **CI** — one `npx playwright test` command — and sharding lets us split the suite across CI machines."*

3.3 MEMORY HOOK for the framework blocks

"Config Runs Projects, Pages Keep Fixtures, Tests Expect Traces."

- **Config** — playwright.config.ts (brain)
- **Runs** — retries, workers, reporters configured
- **Projects** — chromium / firefox / webkit definitions
- **Pages** — Page Object classes
- **Keep Fixtures** — built-in + custom fixtures (page, context, logged-in page)
- **Tests Expect** — spec files with web-first `expect` assertions
- **Traces** — trace viewer + HTML report + CI

3.4 Why POM in Playwright?

Same reason as Selenium: one class per page, locators live in one place, specs read like business steps. Two extra Playwright points:

1. Page objects receive the `page` (or a **fixture**) in the constructor — the same POM idea, cleaner plumbing.
2. Locators in Playwright are **lazy** — `page.getByRole('button')` doesn't search until you act on it — so creating them in the constructor is free.

```
// pages/LoginPage.ts
import { Page, Locator, expect } from '@playwright/test';

export class LoginPage {
  private readonly username: Locator;
  private readonly password: Locator;
  private readonly loginBtn: Locator;

  constructor(private readonly page: Page) {
    this.username = page.getByLabel('Username');
    this.password = page.getByLabel('Password');
    this.loginBtn  = page.getByRole('button', { name: 'Login' });
  }

  async login(user: string, pass: string) {
    await this.username.fill(user);
    await this.password.fill(pass);
    await this.loginBtn.click();
  }

  async expectLoginSuccess() {
    await expect(this.page).toHaveURL(/dashboard/);
  }
}
```

MEMORY HOOK (same as Selenium): *"Locators live with their page; specs stay clean."*

SECTION 4 — BROWSER vs CONTEXT vs PAGE (the architecture question)

Q: Playwright architecture — Playwright Client, Playwright Server, BrowserDriver, Browser Binaries (know all four by name!)

Understand it: This is the exact "explain the architecture of Playwright" question — four named parts, and a click travels through all of them:

```
Your machine
+-----------------------------+
| PLAYWRIGHT CLIENT           |  the Node.js process running your specs
| (your test code + the       |  calls: page.getByRole('button').click()
|  @playwright/test library)  |
+--------------+--------------+
               |   WebSocket (ONE persistent, two-way pipe)
               v
+-----------------------------+
| PLAYWRIGHT SERVER (driver)  |  a SECOND Node process Playwright launches,
| translates API calls into   |  called "the driver" — the brain near the
| browser-protocol messages   |  browser; holds state, queues commands,
| and pushes events back      |  and emits events (popups, downloads,
+--------------+--------------+   console) back over the same pipe
               |
               |   browser-specific protocol:
               |   Chromium -> CDP (Chrome DevTools Protocol)
               |   Firefox / WebKit -> Playwright's own PATCHED builds
               v                (same-style protocol = one API for all three)
+-----------------------------+
| BROWSER BINARIES            |  the actual browser executables
| chromium-1234, firefox-456, |  downloaded by npx playwright install into
| webkit-789 in the           |  the OS cache folder (ms-playwright),
| ms-playwright cache         |  VERSION-LOCKED to the Playwright version
+-----------------------------+
```

The four parts, one by one:

1. **Playwright Client** — your test process (Node.js): the spec files, fixtures, and the @playwright/test API. This is the only part you ever touch directly.
2. **Playwright Server (the driver)** — when a test launches a browser, Playwright starts this second Node.js process. Your client talks to it over **one persistent WebSocket connection** — versus Selenium, where EVERY command is a separate stateless HTTP request. The pipe stays open both ways, which is why Playwright can "listen" for events (a popup opened, a download started) instead of polling.
3. **BrowserDriver / protocol layer** — inside the server: Chromium is driven via **CDP (Chrome DevTools Protocol)**; Firefox and WebKit are Playwright's own **patched browser builds** that speak a matching protocol — that is the trick that lets one API drive all three engines with the same power.
4. **Browser binaries** — the real browser executables, downloaded separately (not part of the npm package — it would be huge) via `npx playwright install`, stored in the OS cache: `~/.cache/ms-playwright` (Linux), `~/Library/Caches/ms-playwright` (Mac), `%USERPROFILE%\AppData\Local\ms-playwright` (Windows). They are **version-locked to the installed Playwright release** — that's why after upgrading Playwright you must re-run `npx playwright install`.

Two bonus points that impress:

- You can point Playwright at your OWN installed Chrome/Edge instead of the bundled build using `channel: 'chrome'` in the config — handy for testing the browser users actually have.
- Because the server speaks over a pipe, one client can even drive browsers on a REMOTE machine (connect over WebSocket to a remote Grid-like server) — the same architecture scaled out.

SAY THIS: "The Playwright client is my Node test process. It spawns the Playwright server — the driver process — and they communicate over one persistent WebSocket. The server drives the browsers using CDP for Chromium and Playwright's patched Firefox and WebKit builds, whose binaries sit in the ms-playwright cache

folder, version-locked to the installed Playwright version. Compared to Selenium, where every command is a separate stateless HTTP request, that single two-way pipe is why Playwright is fast and can listen to browser events."

MEMORY HOOK: *"Client speaks, Server drives, Binaries execute — one WebSocket all the way."*

Q: Why do we run `npx playwright install` after `npm install`? (the binaries question)

Understand it: The npm package contains only the CODE (client + server). The browsers are hundreds of MB, so they are a separate download into the OS cache folder (ms-playwright). They are Playwright's own patched builds, version-locked — upgrade Playwright, re-run install, otherwise you get "Executable doesn't exist" errors.

MEMORY HOOK: *"npm install brings the brain, playwright install brings the browsers."*

4.1 Understand the nesting dolls (Browser -> Context -> Page)

Three Russian nesting dolls:

- **Browser** — the browser application itself (one Chrome, one Firefox). Starting it is expensive (a real process). One per worker.
- **BrowserContext** — an **incognito-like isolated session** inside that browser: its own cookies, storage, cache. Cheap to create. **Every test gets a FRESH context by default** — this is Playwright's secret for test isolation: no cookie bleeding between tests, ever.
- **Page** — one tab. A context can have many pages (tabs/popups).

```
Browser  (1 per worker — heavy)
  +-- Context  (1 per TEST — isolated cookies, instant)
        +-- Page  (0..n tabs)
```

Why interviewers love it: it explains **why Playwright tests are isolated by default** (new context per test) and **how parallelism is cheap** (many contexts in one browser, no grid setup).

4.2 Manual version (you'll be asked to write it)

```
import { chromium } from '@playwright/test';

const browser = await chromium.launch({ headless: true });   // browser
const context = await browser.newContext();                    // isolated session
const page = await context.newPage();                         // a tab
await page.goto('https://example.com');
await context.close();      // wipes all cookies/storage of this session
await browser.close();
```

MEMORY HOOK: *"Browser is the building, Context is the hotel room, Page is the TV — and every test gets a fresh room."*

SECTION 5 — AUTO-WAITING (the #1 differentiator — know it cold)

5.1 Understand it

In Selenium, you say: "find the element... wait... NOW click, and pray." In Playwright, every action (click, fill, check...) **automatically waits until the element is ready**, where "ready" (the official name is **actionability**) means ALL of:

1. **Visible** — displayed on the page
2. **Stable** — not animating/moving
3. **Receives events** — nothing is covering it
4. **Enabled** — not disabled

Only when all four are true does the action happen; otherwise it retries until the timeout (default 30s, configurable). This is why Playwright specs contain almost no explicit wait code.

5.2 Navigational auto-waits

Playwright also auto-waits for the page lifecycle: `goto` waits for `load`, `click` on a link auto-waits for navigation, `expect` (next section) retries.

5.3 When you DO need explicit waits

```
// wait for a URL (SPA navigations don't reload, so wait for the URL)
await page.waitForURL('**/dashboard');

// wait for an element to appear
await page.getByTestId('results').waitFor();

// wait for a network response (e.g., a search API to finish)
await page.waitForResponse(resp => resp.url().includes('/api/search') && resp.status() === 200);

// a fixed pause — ONLY for debugging, never in real tests
await page.waitForTimeout(3000);
```

SAY THIS (the senior version): "Actions auto-wait for visibility, stability, hit-testing and enabled state. I only write explicit waits for things actions can't know — URL changes in SPAs, specific API responses, or element disappearance — using waitForURL, waitForResponse and expect(...).toBeHidden()."

MEMORY HOOK: *"Every action has a waiter built in; I wait only for what actions can't see — URLs and APIs."*

SECTION 6 — LOCATORS (the modern way — forget XPath-first thinking)

6.1 Understand the philosophy

Selenium culture starts with ID/XPath. Playwright culture starts with **how a USER sees the page**: role, label, text — because those match what humans experience and survive most DOM changes. The priority order to recite:

1. **getByRole** — by ARIA role (`button`, `link`, `textbox`, `checkbox`) + accessible name. THE recommended one.
2. **getByLabel** — form fields by their `<label>` text.
3. **getByPlaceholder** — input by placeholder.
4. **getByText** — by visible text.
5. **getByAltText / getByTitle** — images / title attribute.
6. **getByTestId** — `data-testid` attribute: the explicit escape hatch when nothing semantic is stable.
7. **CSS / XPath** — last resort, same as Selenium.

```
await page.getByRole('button', { name: 'Sign in' }).click();
await page.getByLabel('Email').fill('a@b.com');
await page.getByPlaceholder('Search').fill('laptop');
await page.getByRole('link', { name: 'Forgot password?' }).click();
await page.getByTestId('cart-count').click();        // data-testid="cart-count"
await page.locator('#submit').click();               // CSS
await page.locator('xpath=//button[@type="submit"]').click();
```

Why role-based locators are "better" (say this): they test the accessible tree — the same thing screen readers use — so they passively improve accessibility while surviving markup refactors.

6.2 STRICT MODE (a favorite trick question)

If a locator resolves to **MULTIPLE elements**, Playwright **fails the test immediately** instead of silently using the first one (which is Selenium's dangerous default). You must disambiguate:

```
await page.getByRole('listitem').first().click();
await page.getByRole('listitem').nth(2).click();          // 3rd item (0-based)
await page.getByRole('listitem').last().click();
```

MEMORY HOOK: *"Many matches = immediate error; pick first(), nth(), or filter."*

6.3 Filtering and chaining locators (asked as "how do you handle a row in a table?")

```
// filter by text
await page.getByRole('listitem').filter({ hasText: 'Laptop' }).click();

// filter by child locator: the row that CONTAINS a cell 'Laptop'
await page.getByRole('row', { name: 'Laptop' }).getByRole('button', { name: 'Delete' }).click();

// AND-combining
await page.getByRole('button', { name: 'Add' }).and(page.getByRole('button', { name: /cart/i })).click();   // rare

// OR-combining
await page.getByLabel('Username').or(page.getByLabel('Email')).fill('a@b.com');
```

Chaining reads naturally: **row that contains "Laptop" -> its Delete button.** This is the modern replacement for XPath axes (`following-sibling` etc.).

MEMORY HOOK: *"Filter by what it HAS, chain by what it CONTAINS."*

6.4 Shadow DOM (the hidden world inside a component)

Understand it — the sealed drawer in the wardrobe:

The normal DOM is a wardrobe where every drawer is visible — any XPath or CSS can walk anywhere it likes. But modern web components (the YouTube player, GitHub's copy buttons, Salesforce Lightning, every Lit/Polymer/Stencil library) build their internals inside a **shadow root** — a hidden, sealed DOM tree attached to a "host" element. From outside, DevTools and locators see only the host tag; the buttons, spans and inputs living inside it are invisible to normal traversal. Why does it exist? **Encapsulation** — the component's HTML and CSS can't leak out and clash with your page, and your page's global CSS can't corrupt the component. That's why component libraries love it — and why locators break on it.

```
What the page shows you (light DOM)      What is actually inside (shadow DOM)

<video-player id="player">               #shadow-root (open)
    <!-- nothing to see here -->             <div class="controls">
</video-player>                                 <button class="play"></button>
                                                <input class="volume">
                                            </div>
```

Open vs closed — the two flavours a component can choose:

Type	Created with	Who can see inside
open	`attachShadow({ mode: 'open' })`	the page's JS via `element.shadowRoot` — and Playwright
closed	`attachShadow({ mode: 'closed' })`	nobody — even the page's own JS gets `null` from `element.shadowRoot`

Most real-world components (YouTube, GitHub, Lit-based design systems) use **open**.

How Playwright handles OPEN roots — you do NOTHING:

```
// the .play button lives inside TWO nested shadow roots — Playwright still finds it
await page.locator('video-player .play').click();
await page.locator('#player button.play').click();
await page.locator('video-player input.volume').fill('80');
```

No `getShadowRoot()`, no JS executor, no special flag, no per-level walking. CSS (and text) locators pierce every open shadow root — and nesting doesn't matter: a root inside a root inside a root is still transparent to Playwright.

The one trap — interviewers LOVE this follow-up: XPath does NOT pierce shadow DOM in Playwright. `page.locator('xpath=//video-player//button')` sees only the light DOM and finds nothing inside the host. The moment a component has shadow internals, switch to CSS.

CLOSED roots — the honest answer (say this and you sound senior):

Nobody pierces closed — Playwright, Selenium 4, Cypress, nobody — because even the page's own JavaScript cannot reach in (`element.shadowRoot` returns `null`). Two things teams do in practice:

```
// Plan A (the professional one): ask devs for a test hook, or build with
// mode: 'open' in test environments. Testing strategy, not hacking.

// Plan B (the injection trick — flip closed to open before the page loads):
await page.addInitScript(() => {
  const original = Element.prototype.attachShadow;
  Element.prototype.attachShadow = function (init) {
    const root = original.call(this, { ...init, mode: 'open' }); // force open
    this.__shadowRoot = root;                                    // keep a handle
    return root;
  };
});
// now normal CSS locators pierce even the "closed" components
```

The same problem in Selenium 4 (for the comparison question):

```
// Selenium: pierce ONE level at a time, manually
WebElement host = driver.findElement(By.cssSelector("video-player"));
SearchContext shadow = host.getShadowRoot();      // one level only
shadow.findElement(By.cssSelector(".play")).click();
// nested roots? call getShadowRoot() again on each inner host — a chain.
// XPath? no. Closed? no. THIS is the "getShadowRoot() dance".
```

Where you will actually meet it in real projects: the YouTube player (a huge shadow tree), GitHub's clipboard buttons, Salesforce Lightning components, Chrome's built-in `<input type="date">` calendar popup, and any Lit / Polymer / Stencil / Fast design system.

MEMORY HOOK: *"Shadow DOM = a sealed drawer. Playwright owns the x-ray machine — CSS sees through open drawers, even nested ones. XPath left its x-ray at home. A closed drawer blinds everyone — flip it open with an attachShadow initScript."*

SAY THIS (when asked "how do you handle shadow DOM?"):

"Shadow DOM is encapsulated HTML inside a host element — the page shows you `<my-player>` but not the buttons inside it. In Playwright, open shadow roots are pierced automatically: my normal CSS locators just work, even through multiple nested roots — no getShadowRoot() chain like Selenium 4, where I'd have to enter one level at a time and can't use XPath at all. One nuance I'm careful about: in Playwright too, XPath selectors don't pierce shadow DOM, so I stay on CSS. Closed roots are sealed for everyone — even the page's own JavaScript gets null — so my approach there is to ask developers for a test hook or use an open mode in test builds; as a last resort you can override attachShadow with an addInitScript to force the roots open."

SECTION 7 — WEB-FIRST ASSERTIONS (the expect() system)

7.1 Understand it

Selenium's `Assert.assertEquals(actual, expected)` checks ONCE — right now — so any timing mistake fails the test. Playwright's web-first assertions **retry until the condition becomes true or the timeout ends**:

```
import { test, expect } from '@playwright/test';

await expect(page.getByRole('heading')).toHaveText('Welcome, Jashu');
await expect(page.getByTestId('cart-count')).toHaveText('3');
await expect(page.getByRole('button', { name: 'Save' })).toBeVisible();
await expect(page.getByRole('button', { name: 'Save' })).toBeEnabled();
await expect(page.getByRole('alert')).toContainText('Order placed');
await expect(page.locator('tr')).toHaveCount(5);
await expect(page).toHaveTitle(/Dashboard/);
await expect(page).toHaveURL(/dashboard/);
await expect(page.getByRole('checkbox')).toBeChecked();
await expect(page.getByRole('textbox')).toHaveValue('hello');
await expect(page).toHaveScreenshot();              // visual regression
```

The naming tells the story: `toBeVisible` (asserting the CURRENT state is visible — retries), vs `page.getByX().isVisible()` (returns true/false NOW, no retry — used inside `if` conditions, not for assertions). Interviewers test exactly this difference — say: "I assert with `await expect(...)` and only use `.isVisible()` as an instantaneous boolean check."

MEMORY HOOK: *"expect() waits and retries; .isVisible() just looks and answers."*

7.2 expect.soft — Playwright's SoftAssert

```
await expect.soft(page.getByTestId('err1')).toBeVisible();
await expect.soft(page.getByTestId('err2')).toBeVisible();   // failures collect...
// ...test continues, all failures reported at the end
```

Same idea as TestNG's SoftAssert — use it when you want ALL the validation errors, not just the first.

7.3 Negative assertions

`toBeHidden()`, `not.toBeVisible()`, `toHaveCount(0)` — and know that waiting for something to disappear (`await expect(el).toBeHidden()`) is the correct pattern after closing a spinner.

SECTION 8 — FIXTURES (the plumbing question — "where does page come from?")

8.1 Understand it

A fixture is a **ready-to-use object the runner hands to your test** — like a hotel service: you don't build the shower, the room comes with one. `test('...', ({ page }) => {...})` — you never created a browser or a page. The runner did, and it will also **clean them up automatically** after the test (even if it fails or throws — Playwright's garbage collector).

Built-in fixtures you get for free:

- `page` — an isolated page in a fresh context (the one you use 90% of the time)
- `context` — the test's BrowserContext (when you want multiple tabs)
- `browser` — the browser instance (when you need raw control)
- `browserName` — 'chromium' | 'firefox' | 'webkit' — to branch per project
- `request` — an API client for API testing/calls
- `testInfo` — metadata: title, retries left, annotations

```
import { test, expect } from '@playwright/test';

test('login works', async ({ page }) => {
  await page.goto('/login');          // baseURL from config is prepended!
  await page.getByLabel('Username').fill('admin');
  ...
});
```

8.2 Custom fixtures — the senior answer (test.extend)

The problem: every spec needs a **LoginPage** object, or an **already-logged-in page**. Copy-pasting setup into every file = the old POM constructor mess.

The fix: extend the fixtures once, and every test can request them like they were built-in:

```
// fixtures/pages.ts
import { test as base, Page } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';

// declare what we're adding
type Pages = {
  loginPage: LoginPage;
};

export const test = base.extend<Pages>({
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page));   // hand it to the test, cleanup after
  },
});
export { expect } from '@playwright/test';

// now in the spec:
import { test, expect } from '../fixtures/pages';

test('user can login', async ({ loginPage }) => {
  await loginPage.login('admin', 'admin123');
  await expect(loginPage.page).toHaveURL(/dashboard/);
});
```

Why this is better than constructors (say this): "Fixtures give lazy setup and automatic teardown per test — the object is created only when a test asks for it, and the runner handles cleanup even on failure. It's dependency injection for tests."

8.3 Overriding built-in fixtures

You can also override `page` itself — e.g., to apply storageState, or mock geolocation for EVERY test — in one place (config `use` section) or via a custom fixture. This is how "already logged in" pages work at scale (Section 12).

MEMORY HOOK: *"Fixtures are hotel services — order them by name in ({ }), the runner builds them and cleans them up."*

SECTION 9 — HOOKS, TAGS & TEST CONTROL

9.1 The hooks (vs TestNG's before/after)

```
import { test, expect } from '@playwright/test';

test.describe('Checkout flow', () => {

  test.beforeEach(async ({ page }) => {       // before EVERY test — like @BeforeMethod
    await page.goto('/login');
    await login(page, 'admin', 'admin123');  // fresh login state per test
  });

  test.afterEach(async ({ page }) => { /* cleanup, evidence */ });

  test.beforeAll(async () => { /* once per worker/file — like @BeforeClass */ });
  test.afterAll(async () => { /* once per worker/file */ });

  test('adds item to cart', async ({ page }) => { ... });
  test('removes item', async ({ page }) => { ... });
});
```

Mapping table to recite if you come from TestNG:

- `beforeEach` = @BeforeMethod • `afterEach` = @AfterMethod
- `beforeAll` = @BeforeClass • `afterAll` = @AfterClass
- There is NO beforeSuite/afterSuite — the **config + projects** replace the suite concept.

CAREFUL (interviewers trap here): `beforeAll` runs once **per worker**, and since each test gets a FRESH context, you can't store a logged-in `page` from beforeAll for the tests. Use storageState (Section 12) or beforeEach for login.

9.2 Tags, only, skip, fixme, and grep

```
test('critical journey @smoke', async ({ page }) => { ... });

test.only('just this one while debugging', ...);   // .only = run only this
test.skip(browserName === 'webkit', 'no webkit support yet');
test.fixme('known product bug #2231');             // known-broken, not failed
```

Run subsets: `npx playwright test --grep "@smoke"` — the modern replacement for testng.xml groups.

9.3 Serial mode (and why it's a last resort)

By default tests in a file run in PARALLEL across workers. If tests genuinely depend on each other:

```
test.describe.configure({ mode: 'serial' });   // run in order, stop chain on failure
```

But say: "We avoid serial — it hides state dependencies. Our rule is every test can run alone." (This sentence is interview gold.)

MEMORY HOOK: *"Each = method, All = class, there's no suite — config is the suite."*

SECTION 10 — playwright.config.ts (the brain — "explain your config")

10.1 A real config, line by line

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './src/tests',              // where specs live
  fullyParallel: true,                 // even tests inside ONE file run in parallel
  retries: process.env.CI ? 2 : 0,      // retry flaky tests twice in CI only
  workers: process.env.CI ? 4 : 6,      // parallel processes (each with own browser)
  reporter: [
    ['html', { open: 'never' }],       // playwright-report/index.html
    ['json', { outputFile: 'results.json' }],
    ['list']                           // console output
  ],
  use: {
    baseURL: 'https://staging.myapp.com',   // page.goto('/login') works!
    screenshot: 'only-on-failure',          // automatic evidence
    trace: 'retain-on-failure',              // trace.zip recorded, kept on failure
    video: 'off',
    actionTimeout: 15_000,                   // default per-action wait
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit',  use: { ...devices['Desktop Safari'] } },
    { name: 'mobile',  use: { ...devices['iPhone 14'] } },   // emulation!
  ],
});
```

10.2 The concepts each line carries (be able to expand any of them)

- **projects** — cross-browser/cross-device matrix. Each spec file runs once per project -> 4 projects × 10 specs = 40 test runs, all parallel. THE replacement for testng.xml + Grid.
- **workers** — number of parallel OS processes, each with its own browser. Playwright scales out of the box — **no Grid needed**. Say: "By default workers = CPU cores / 2; we tune it for CI."
- **retries** — auto-rerun failed tests (Playwright marks them as `flaky` in the report — a beautiful detail to mention).
- **reporter** — html (default), list, json, junit — or Allure/Slack via community packages.
- **use** — default options injected into every test: `baseURL`, `trace`, `screenshot`, `video`, `actionTimeout`, storageState, geolocation, permissions, locale, timezone.
- **devices** — Playwright's built-in device catalog (iPhone, Pixel, viewports, touch, user agents) for responsive testing.
- **CLI flags beat config when needed:** `--headed`, `--project=chromium`, `--grep=@smoke`, `--workers=2`, `--update-snapshots`.

> **MEMORY HOOK:** *"Config answers the 5 Ws: WHERE (baseURL), WHO (projects), HOW MANY (workers), WHO AGAIN (retries), WHAT EVIDENCE (trace/screenshot/reporter)."*

SECTION 11 — THE END-TO-END FLOW: CONFIG -> HTML REPORT (including test data)

11.1 Understand the full journey of one run

```
Developer / CI runs:  npx playwright test
   |
   v
1. PLAYWRIGHT RUNNER starts
   • reads playwright.config.ts
   • loads all *.spec.ts files from testDir
   • scans for test() definitions (no execution yet — just discovery)
   |
   v
2. SCHEDULING
   • multiplies tests × projects (chromium, firefox...)
   • splits them across WORKERS (each worker = own process + own browser)
   • sharding: --shard=1/3 lets CI machines take a slice each
   |
   v
3. PER TEST, the fixtures cascade (auto setup):
   browser (per worker) -> fresh context (per test!) -> page (per test)
   • test data loaded here: JSON fixtures / CSV readers / storageState file
   |
   v
4. beforeEach hook -> e.g., go to /login (baseURL applied)
   |
   v
5. TEST BODY
   • page-object methods: locators + auto-waiting actions
   • web-first expect() assertions — retrying until true
   • traces/screenshots recorded if configured (on-failure)
   |
   v
6. afterEach -> cleanup; fixtures torn down (context closed = cookies wiped)
   |
   v
7. FAIL? -> retried if retries > 0 -> second attempt marked 'flaky' if it passes
   |
   v
8. ALL WORKERS finish -> reporters fire
   • playwright-report/index.html generated
   • test-results/ holds traces/videos/screenshots of failures
   |
   v
9. Debug a failure:  npx playwright show-trace test-results/.../trace.zip
   -> step-by-step DOM snapshots, console, network, timing — a flight recorder
   |
   v
10. CI publishes the HTML report as an artifact / serves it; team reviews
```

11.2 Where TEST DATA enters (3 doors — same pattern as Selenium)

Door	Source	What it holds	Example
1. Environment	config `use.baseURL`	same for all tests	staging URL
2. Input data	JSON/CSV imported by spec or fixture	rows per test	users.json read in a loop
3. Session data	`.auth/*.json` storageState	logged-in cookies	skip login in every test

SAY THIS (60-second spoken flow): "The runner reads playwright.config.ts, discovers all spec files, and multiplies each test by its projects, then distributes them across parallel workers — each worker with its own browser. For every test the fixtures give a fresh isolated context and page, my beforeEach navigates with the configured baseURL, and the test drives page objects with auto-waiting actions and web-first assertions. Data comes from JSON fixtures or a storageState for the logged-in session. On failure, traces and screenshots are captured automatically, retries rerun flaky tests, and at the end the HTML reporter generates the report — which I open, or debug deeper with the trace viewer."

MEMORY HOOK: *"Read -> Discover -> Schedule -> Isolate -> Assert -> Retry -> Report."*

SECTION 12 — TEST ISOLATION & AUTH (storageState — asked in every serious interview)

12.1 Understand the problem

Every test = fresh context = **no cookies = no session**. If every test logs in via UI, your suite is slow and you're re-testing login 200 times.

12.2 The fix: login ONCE, save the session, reuse it

```
// Step 1: a setup "project" (a special spec that only logs in)
// tests/auth.setup.ts
import { test as setup } from '@playwright/test';

setup('authenticate', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Username').fill('admin');
  await page.getByLabel('Password').fill('admin123');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await page.waitForURL('/dashboard');

  await page.context().storageState({ path: '.auth/user.json' });  // save cookies + localStorage
});

// Step 2: config — run setup first, then use the state in real tests
export default defineConfig({
  projects: [
    { name: 'setup', testMatch: /auth\.setup\.ts/ },
    {
      name: 'chromium',
      dependencies: ['setup'],                          // run setup first!
      use: { storageState: '.auth/user.json' },           // tests start LOGGED IN
    },
  ],
});
```

Now every test begins already authenticated — the login is performed once per run, at API speed, and each test still gets its OWN isolated copy of the session.

Senior add-ons: one storageState file per ROLE (admin.json, customer.json); or authenticate purely via API: `request.post('/api/login')` and save the token — no UI at all. Also `session` APIs exist for signed-in patterns; know the word **storageState** cold.

MEMORY HOOK: *"Login is a setup project, saved as storageState — tests inherit the session, not the test time."*

SECTION 13 — TABS, DIALOGS & IFRAMES (scenario corner)

13.1 Multiple tabs/popups

Playwright **listens** for new pages instead of hunting window handles:

```
const [popup] = await Promise.all([
  page.waitForEvent('popup'),          // listen first...
  page.getByRole('link', { name: 'Terms' }).click(),   // ...then trigger
]);
await popup.getByRole('heading').waitFor();
await popup.close();                   // back on the original page
```

`Promise.all` = "listen and click at the same time." The same pattern works for `waitForEvent('download')` and `waitForEvent('console')`.

Q: 4 tabs open — how do you come back to the ORIGINAL tab? (bringToFront + context.pages — multi-tab in depth)

Understand the BIG difference from Selenium first: in Selenium there is ONE driver aimed at ONE "active window" — that's why you must `switchTo().window(handle)` before every action or you hit the wrong tab. In Playwright **every tab is its own Page object**, bound to its tab forever — there is NO switching at all. You act on whichever Page variable you want, and **actions work even on background tabs**.

So what does `bringToFront()` actually do? Only this: it makes the tab the **visually front** one in the browser window — for humans, screenshots, and headed debugging. It does NOT aim your code at that tab. `await tab2.getByRole('button').click()` works whether tab2 is in front or buried behind three others. Targeting in Playwright = which Page object you call, nothing else.

```
test('four tabs', async ({ page, context }) => {
  await page.goto('https://example.com');
  const original = page;                    // keep the reference to the ORIGINAL tab

  const twitter = await context.newPage();  // TAB 2 — opened programmatically
  await twitter.goto('https://x.com');
  const [linkedin] = await Promise.all([     // TAB 3 — from a link click (target=_blank)
    context.waitForEvent('page'),
    page.getByRole('link', { name: 'LinkedIn' }).click(),
  ]);
  const [docs] = await Promise.all([       // TAB 4
    context.waitForEvent('page'),
    page.getByRole('link', { name: 'Docs' }).click(),
  ]);

  const allTabs = context.pages();                     // ALL open tabs as an array
  const payTab = context.pages().find(p => p.url().includes('payment')); // find by URL

  await twitter.getByRole('button', { name: 'Follow' }).click(); // background-safe
  await twitter.bringToFront();                        // cosmetic — just for eyes

  // COME BACK TO THE ORIGINAL — you never actually LEFT it:
  await original.bringToFront();                       // visually front again
  await original.getByRole('link', { name: 'Home' }).click(); // acts on tab 1
  // bonus: docs.opener() returns the tab that OPENED docs (here: original)
});
```

Three points interviewers listen for:

1. **No switchTo exists** — each tab is a Page object; you call methods on the reference you want.
2. **bringToFront() is cosmetic** (visual activation), never targeting.
3. **context.pages()** lists all tabs; `.find(p => p.url()...)` recovers a tab whose reference you lost; `tab.close()` closes one; `page.opener()` gets the parent tab.

Also note: clicking a `target="_blank"` link on `page` does NOT change what `page` refers to — the new tab is a separate Page you capture with `waitForEvent('page')` (or `context.waitForEvent('page')` when several tabs may spawn).

MEMORY HOOK: *"Selenium switches, Playwright points — bringToFront is just for eyes."*

SAY THIS: "In Playwright every tab is its own Page object, so I never switch. I keep the original page reference, capture new tabs with waitForEvent('page') or context.newPage(), act directly on any tab — even background ones — list tabs with context.pages(), and come back to the original simply by using its reference again, with bringToFront just for visibility."

Q: 5 tabs are open and I am on the 5th tab — how do I go to the 2nd or 3rd tab?

Understand it — first, unlearn the Selenium reflex: in Playwright you are never really "ON" a tab. There is no active-window pointer anywhere in the API — your code acts on whichever Page object you hold a reference to, even if that tab is buried behind four others. So "going to the 2nd tab" reduces to ONE thing: getting the right Page object. Once you have it, you can act on it immediately; `bringToFront()` only makes it the visually front tab (for your eyes, screenshots, and headed debugging). Three ways to get that object, depending on what you know about the tab.

Way 1 — by POSITION: `context.pages()` is an array

```
const tabs = context.pages();   // ALL open tabs, in open order
// tabs[0] = the first tab (your original page), tabs[4] = the 5th tab

const second = tabs[1];         // the 2nd tab
const third  = tabs[2];         // the 3rd tab

await third.bringToFront();     // visually front — for eyes/screenshots
await third.getByRole('button', { name: 'Pay' }).click();  // act on it
```

The trap with positions: indexes reflect the CURRENT open order. Close tab 2, and the old tab 3 becomes `tabs[1]` — every later tab shifts left. Position works when you JUST opened the tabs and know the state; it is NOT a stable identity across closes.

Way 2 — by IDENTITY: find it by URL or title (the stable, interview-first answer)

```
// by URL — the workhorse
const payTab = context.pages().find(t => t.url().includes('/payment'));

// by title — when the URL is dynamic
const byTitle = context.pages().find(t => (await t.title()) === 'Offers');

if (!payTab) throw new Error('payment tab not open!');   // find() can return undefined

await payTab.bringToFront();
await payTab.getByRole('button', { name: 'Confirm' }).click();
```

This survives tabs opening, closing and reordering — you ask for the tab by WHO IT IS, not by where it sits. Give this answer FIRST in an interview, then show Way 1 as the quick-and-dirty alternative.

Way 3 — FROM BIRTH: keep references as tabs open (never lose them at all)

```
// when a click opens exactly one new tab:
const [newTab] = await Promise.all([
    context.waitForEvent('page'),
    page.getByRole('link', { name: 'Open report' }).click(),
]);

// when SEVERAL tabs may spawn from one action: listen at the context level
const opened: Page[] = [];
context.on('page', tab => opened.push(tab));   // every new tab lands in my list

await page.getByRole('link', { name: 'Open all' }).click();   // spawns 3 tabs
await opened[2].bringToFront();   // the 3rd tab that action opened
```

Working with the whole set (the follow-ups they extend with):

```
// list what is open — your getWindowHandles equivalent, but rich
for (const t of context.pages()) {
    console.log(await t.title(), '->', t.url());
}
```

```
// close every tab EXCEPT the original
for (const t of context.pages()) {
    if (t !== page) await t.close();
}

// a background tab must FINISH loading before you act — wait on ITS state
await third.waitForLoadState('load');
```

The Selenium comparison (say it — it scores): the same question there is genuinely harder — `getWindowHandles()` returns a Set with no reliable order, so you either convert it to a List and index it, or loop and match by title/URL before each `switchTo()`. Playwright replaces that whole dance with `context.pages().find(t => t.url()...)`.

MEMORY HOOK: *"You are never ON a tab in Playwright — you HOLD one. By position if fresh, by URL if stable, by reference if smart — and bringToFront is just for eyes."*

SAY THIS: "In Playwright there's no active-tab concept — every tab is a Page object, so 'going to the 2nd tab' means getting the right reference. If I know the state I just created, context.pages() gives me the array and I take index 1 or 2. If tabs come and go, I find them by URL or title — that's stable. And best practice is to capture references at birth with waitForEvent page or a context page listener, so I never search at all. bringToFront is only for visibility — my actions work on background tabs directly, which is something Selenium's single active window can't do."

13.2 JavaScript dialogs (alert/confirm)

Playwright **auto-dismisses** dialogs by default. To accept or read them, register a handler BEFORE the trigger:

```
page.once('dialog', async d => {
  console.log(d.message());
  await d.accept();                    // or d.dismiss()
});
await page.getByRole('button', { name: 'Delete' }).click();
```

13.3 Iframes — frameLocator (the easiest of any tool)

```
const editor = page.frameLocator('#editor-frame');      // enter the frame's world
await editor.getByLabel('Comment').fill('Hello');        // locator MAGICALLY scopes inside
await editor.getByRole('button', { name: 'Post' }).click();
```

No switchTo, no coming back — the frameLocator IS the scope. Nested frames chain: `page.frameLocator('#outer').frameLocator('#inner')`.

MEMORY HOOK: *"Tabs: listen with Promise.all. Dialogs: handler before click. Frames: frameLocator is the world."*

SECTION 14 — NETWORK: MOCKING, INTERCEPTING, API TESTING

14.1 Understand it

Playwright can see and control EVERY network call the page makes — over that WebSocket pipe. This enables three interview favorite skills:

14.2 Intercept & mock with page.route (the #1 skill)

```
// MOCK a failing API — test the UI's error handling without breaking anything
await page.route('**/api/orders', route =>
  route.fulfill({
    status: 500,
    contentType: 'application/json',
    body: JSON.stringify({ error: 'Server down' }),
  })
);
await page.getByRole('button', { name: 'My Orders' }).click();
await expect(page.getByRole('alert')).toContainText('Something went wrong');

// DELAY/abort — simulate slow internet or blocked API
await page.route('**/api/analytics', route => route.abort());

// MODIFY a real response (part real, part fake)
await page.route('**/api/user', async route => {
  const response = await route.fetch();            // real call
  const body = await response.json();
  body.name = 'Test User';                          // tamper
  await route.fulfill({ response, body: JSON.stringify(body) });
});
```

Say this: "route + fulfill = mock; route.fetch + modify = partial mock; route.abort = block. I mock third-party APIs and failure states that are impossible to produce in a real environment."

14.3 API testing with the request fixture

```
test('API: create order', async ({ request }) => {
  const resp = await request.post('/api/orders', {
    data: { item: 'laptop', qty: 1 },
    headers: { Authorization: `Bearer ${token}` },
  });
  expect(resp.status()).toBe(201);
  expect(await resp.json()).toMatchObject({ status: 'created' });
});
```

One tool for UI AND API — say: "We do mixed tests: create data via API (fast), verify it in the UI, clean up via API."

14.4 Waiting for network

`await page.waitForResponse(resp => resp.url().includes('/api/search') && resp.status() === 200)` — used when the UI updates after a specific API returns. Also `page.route` with HAR files for recorded/replayed traffic.

MEMORY HOOK: *"route.fulfill = fake it, route.fetch = shape it, route.abort = block it, request = be the client."*

SECTION 15 — EMULATION & VISUAL TESTING (a Playwright specialty)

```
// Mobile device emulation (built-in device catalog)
{ name: 'mobile', use: { ...devices['iPhone 14'] } }

// Or per-test via fixtures:
test.use({ viewport: { width: 390, height: 844 } });
test.use({ geolocation: { latitude: 19.07, longitude: 72.87 } });   // Mumbai
test.use({ permissions: ['geolocation'] });
test.use({ locale: 'en-IN', timezoneId: 'Asia/Kolkata' });
test.use({ colorScheme: 'dark' });
test.use({ offline: true });                                    // offline app testing
```

Visual regression — Playwright's own image comparison:

```
await expect(page).toHaveScreenshot('dashboard.png');        // whole page
await expect(page.getByRole('navigation')).toHaveScreenshot();  // one element
// first run: npx playwright test --update-snapshots  (creates the baseline)
```

On mismatch the test fails and the report shows actual vs expected vs DIFF images.

MEMORY HOOK: *"devices for phones, use{} for the rest, toHaveScreenshot for pixel-perfect."*

SECTION 16 — FILES, UPLOADS, DOWNLOADS, INPUTS (daily scenario bag)

```
// UPLOAD — no dialog, no Robot class: point at the input
await page.setInputFiles('input[type=file]', 'test-data/receipt.pdf');

// DOWNLOAD
const [download] = await Promise.all([
  page.waitForEvent('download'),
  page.getByRole('link', { name: 'Export CSV' }).click(),
]);
await download.saveAs('downloads/report.csv');

// HOVER, right-click, keyboard
await page.getByRole('menuitem', { name: 'Billing' }).hover();       // opens submenus
await page.getByRole('row', { name: 'Laptop' }).click({ button: 'right' });
await page.keyboard.press('Control+A');
await page.getByRole('textbox').pressSequentially('sel', { delay: 100 }); // type like a human

// CHECKBOX / RADIO — and their smart assertions
await page.getByRole('checkbox').check();
await expect(page.getByRole('checkbox')).toBeChecked();

// DRAG & DROP
await page.getByTestId('draggable').dragTo(page.getByTestId('dropzone'));
```

MEMORY HOOK: *"Upload = setInputFiles; Download = waitForEvent + saveAs; Drag = dragTo."*

SECTION 17 — DEBUGGING TOOLKIT (asked as "how do you debug failures?")

Five tools, in the order to recite them:

1. **Trace Viewer** — the killer feature. On failure, `trace.zip` records a full flight-recorder: every action with DOM snapshot, network, console, timing. `npx playwright show-trace trace.zip`. Say: "It's like a black box of the test — I can rewind the DOM at any step."
2. **UI Mode** — `npx playwright test --ui`: a watch-mode app with a time-travel explorer, filters by project/grep, run one test, see traces. The modern dev experience.
3. **Playwright Inspector / --debug** — `npx playwright test --debug`: steps through the test line by line, showing the locator it's resolving and the auto-waiting (try `page.pause()`).
4. **Codegen** — `npx playwright codegen https://site.com`: records your clicks and prints BEST-PRACTICE locators (getByRole etc.). Great for discovering locators and learning the style — say: "codegen to discover, hand-write in the spec."
5. **VS Code extension** — run/debug tests from the editor, watch mode, pick locators.

MEMORY HOOK: *"Trace to investigate, UI mode to develop, --debug to step, codegen to discover."*

SECTION 18 — PARALLELISM, RETRIES & FLAKINESS

- **workers** = parallel processes; each with its OWN browser; tests never share a worker with a stale context (fresh context per test anyway).
- **fullyParallel: true** — even tests inside the same file run in parallel (default for new projects). Turn OFF with `fullyParallel: false` or per-describe `test.describe.configure({ mode: 'serial' })` when order truly matters.
- **sharding** — `npx playwright test --shard=1/4` splits the suite across 4 CI machines -> 4× faster wall-clock. No Grid, no nodes.
- **retries** — failed tests rerun; if they pass on retry the report marks them **flaky** (yellow), so you can track down the root cause instead of hiding it.
- The interview line: "Isolation (fresh context per test) + auto-waiting (built into actions) + web-first assertions are Playwright's three flake-killers; retries are the safety net, and flaky-marking keeps us honest."

MEMORY HOOK: *"Workers for processes, sharding for machines, retries for luck, isolation for truth."*

SECTION 19 — REPORTING & CI

- **Reporters:** `html` (default — interactive: filter, attachment view, watch trace inline), `list`, `json`, `junit`. Community: Allure, Slack bots, Microsoft Teams.
- **Artifacts on failure** — config's `trace: 'retain-on-failure'`, `screenshot: 'only-on-failure'`, `video: 'retain-on-failure'` — automatic evidence, no listener code needed (vs Selenium: ITestListener + TakesScreenshot!). Great contrast line for interviews: "In Selenium we wrote listeners to screenshot on failure; in Playwright it's a config line."
- **CI flow (SAY THIS):** "On pull request, CI installs dependencies, runs `npx playwright install --with-deps`, runs `npx playwright test` sharded across machines, and publishes the HTML report and traces as build artifacts. Failures fail the pipeline; the report link lands in the PR/Slack."
- GitHub Actions has an official `microsoft/playwright-github-action`.

SECTION 20 — DATA-DRIVEN TESTING (Playwright style)

```
// 1. Loop over rows — same @DataProvider idea, plain JS
for (const { user, pass, expected } of users) {           // users from JSON
  test(`login as ${user}`, async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Username').fill(user);
    await page.getByLabel('Password').fill(pass);
    await expect(page.getByRole('alert')).toContainText(expected);
  });
}
// NOTE: build the test INSIDE the loop (dynamic titles) — a known interview gotcha.

// 2. Fixture-based data (computed/lazy per test)
const test = base.extend<{ userData: User }>({
  userData: async ({}, use) => use(generateRandomUser()),
});

// 3. API-created data for setup, API cleanup after (fast + isolated)
```

MEMORY HOOK: *"Rows from JSON, randomness from fixtures, setup and cleanup by API."*

SECTION 21 — COMMANDS & VERSIONS (know these cold)

```
npm init playwright@latest          # scaffold project
npm i -D @playwright/test           # the dependency (current line: 1.62.x, Sept 2026)
npx playwright install              # download browsers (chromium, firefox, webkit)
npx playwright install chromium     # just one
npx playwright test                  # RUN the suite
npx playwright test --headed         # see the browser
npx playwright test --project=chromium --grep @smoke
npx playwright test --ui             # UI mode
npx playwright test --debug          # inspector
npx playwright codegen https://site.com
npx playwright show-trace trace.zip   # view a trace
npx playwright show-report           # open the HTML report
```

21.1 The package.json — required packages with versions (the interview table)

```
{
  "devDependencies": {
    "@playwright/test": "^1.62.0",
    "typescript": "~5.9.0",
    "@types/node": "^24.0.0",
    "allure-playwright": "^3.10.0",
    "@faker-js/faker": "^10.6.0",
    "winston": "^3.19.0",
    "dotenv": "^17.2.0"
  },
  "scripts": { "test": "playwright test" }
}
```

What each one is for — with the Selenium-world equivalent (interviewers LOVE this mapping):

npm package	Version line	Purpose	Selenium-world equivalent
`@playwright/test`	1.62.x	the runner + automation engine	selenium-java + TestNG + Surefire
`typescript` / `@types/node`	~5.9 / ^24	the language + Node typings	(Java is the language already)
`allure-playwright`	3.10.x	Allure reporting (step-by-step report)	Extent Reports / allure-testng
`@faker-js/faker`	10.6.x	random test data generator (names, emails, cards)	Java Faker
`winston`	3.19.x	logging to console + files	Log4j2
`dotenv`	17.x	.env files for URLs/credentials per environment	config.properties

Notes that earn points:

- The **built-in HTML reporter needs NO dependency** — it ships with @playwright/test. allure-playwright is the optional upgrade for the classic Allure report.
- **Test data:** faker generates unique data per run (unique emails = no clashing tests) — the modern replacement for shared Excel rows.
- **Credentials/URLs:** dotenv loads a `.env` file into `process.env` — the direct equivalent of reading config.properties.
- Runtime to mention: **Node 20 LTS** (current Playwright lines need Node 18+).
- If asked "which version?" — state the number, then "we keep it updated; Playwright ships roughly monthly and we follow the latest stable line."

MEMORY HOOK: *"Runner, Reporter, Random-data, Logger, Env — R-R-R-L-E."*

21.2 Winston — the logging library (the Log4j of the Node/Playwright world)

Understand it: Playwright has no built-in logging framework (console.log is not a logging system). The standard answer in Node projects is **winston** — and it maps 1:1 onto Log4j concepts:

Winston concept	Log4j equivalent
levels: error, warn, info, http, verbose, debug, silly	ERROR, WARN, INFO, DEBUG, TRACE
transports (Console, File, daily-rotate)	**appenders** (ConsoleAppender, FileAppender, RollingFile)
`createLogger({...})`	log4j2.xml configuration

```
import winston from 'winston';

export const logger = winston.createLogger({
  level: 'info',                                    // minimum level that gets logged
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.printf(({ timestamp, level, message }) =>
      `${timestamp} [${level}] ${message}`)
  ),
  transports: [
    new winston.transports.Console(),                  // like a ConsoleAppender
    new winston.transports.File({ filename: 'logs/test.log' }),  // like a FileAppender
  ],
});

// in a test or page object:
logger.info(`Logging in as ${user}`);
logger.error(`Payment failed: ${error}`);
```

SAY THIS: "For logging we use winston — levels from error up to silly, and transports for console and file, which maps directly onto Log4j's levels and appenders. The built-in HTML reporter covers reporting, and we add allure-playwright when the team wants Allure."

MEMORY HOOK: *"Levels error->silly, transports = appenders, one logger many destinations."*

SECTION 22 — SCENARIO QUESTIONS CORNER (every scenario, fully explained)

Q: The element is found but the click fails. How do you handle it?

Understand it: Playwright auto-waits before EVERY action — the element must be visible, stable (not animating), receive events (not covered), and enabled. So when a click still fails, one of those four actionability checks is failing, and each has a different fix. This is a "do you understand what auto-waiting actually checks" question in disguise.

What to check, in order:

```
// 1. Give the auto-wait more room — default is 30s per action; heavy pages need more
await page.getByRole('button', { name: 'Submit' }).click({ timeout: 15_000 });

// 2. A spinner/banner is covering it — wait for the cover to leave first
await page.locator('.loading-spinner').waitFor({ state: 'hidden' });

// 3. It's animating — tell the page to calm down (config-level, better)
//    use: { reducedMotion: 'reduce' }  in playwright.config.ts

// 4. LAST RESORT — force skips ALL actionability checks; use to prove a bug fast
await page.getByRole('button', { name: 'Submit' }).click({ force: true });
```

When to admit `force`: it works every time because it checks nothing — which is exactly why it's a debugging tool, not a habit. Say that sentence in the interview and you sound senior.

SAY THIS: "If a click fails despite auto-waiting, one of the four actionability checks is failing — covered, animating, hidden, or disabled. I wait for the covering element to disappear, increase the timeout, or enable reduced-motion in config for animation-heavy pages. Force-click is my last resort and only to isolate the problem, because it skips the very checks that prove a real user could click."

Q: How do you test SPA navigation (React/Angular pages that never reload)?

Understand it: A SPA changes screens WITHOUT a page load — the URL changes, the DOM updates, but the `load` event you'd wait for in Selenium never fires again. Waiting for "page load" in an SPA is waiting for something that already happened. The assertion must follow the thing that actually changes: the URL or the element.

```
await page.getByRole('link', { name: 'Dashboard' }).click();

// assert the thing that actually changed — the URL
await expect(page).toHaveURL(/.*dashboard/);
// or: await page.waitForURL('**/dashboard');

// assert the new screen's content is real
await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
```

SAY THIS: "In an SPA there's no page load after navigation — the URL and DOM change in place. So I never wait for load events; I assert with toHaveURL or waitForURL, and back it with a visibility assertion on the new screen's key element."

Q: The app generates dynamic IDs. How do you locate elements then?

Understand it: Same disease as Selenium — React/Angular stamp a fresh number into ids (`input-3847`). But Playwright's cure is philosophical: stop locating on implementation details AT ALL. Role-based locators bind to what the element IS (a button named Submit), not to what the framework generated.

```
// the Playwright ladder for unstable ids — from best to last resort
await page.getByRole('button', { name: 'Submit' });          // accessible role + name
await page.getByLabel('Username');                           // label association
await page.getByTestId('submit-btn');                        // stable hook (ask devs!)
await page.locator('[id^="submit_"]');                       // CSS operators as fallback
await page.locator('div').filter({ hasText: 'Laptop' });     // filter by content
```

SAY THIS: "Dynamic ids mean I stop using ids. My first choice is getByRole with the accessible name — it survives any refactor. getByTestId is the stable hook I ask developers to add, CSS operators like id-contains are the fallback — same philosophy as Selenium's contains(), cleaner syntax. The long-term fix is data-testid, not cleverer locators."

Q: Shadow DOM — how do you handle it? (summary — full deep-dive is Section 6.4)

Understand it: Shadow DOM is a sealed capsule inside a host element. Playwright's difference from Selenium: CSS and text locators pierce OPEN shadow roots automatically — even nested ones — with zero extra code.

```
// .play sits inside TWO shadow roots — this just works
await page.locator('video-player .play').click();
// XPath does NOT pierce — switch to CSS when a component has shadow internals
```

Closed roots are sealed for everyone (even the page's own JS gets null) — the professional fix is a test hook, or flipping `mode` to open via an `addInitScript` override on `attachShadow` (full code in Section 6.4).

SAY THIS: "Open shadow roots are transparent to my CSS locators — even nested — no getShadowRoot() chain like Selenium. XPath is the exception: it doesn't pierce, so I stay on CSS. Closed roots are sealed by design, so I'd ask for a test hook or force open mode with an init script in test builds."

Q: How do you read and verify a specific table row?

Understand it: The modern table answer is ROLE-based, not XPath-index-based: a row is a role, its cells are reachable through the row's name. Position-based `tr[3]/td[2]` breaks when data reorders; name-based survives.

```
// the row that BELONGS to Laptop — however many rows exist
const row = page.getByRole('row', { name: 'Laptop' });

// read a specific cell of that row (2nd cell)
const price = await row.locator('td').nth(1).textContent();
await expect(row.getByRole('button', { name: 'Delete' })).toBeVisible();

// verify the row count
await expect(page.getByRole('table').getByRole('row')).toHaveCount(8);

// the delete flow — click, then assert it's GONE
await row.getByRole('button', { name: 'Delete' }).click();
await expect(page.getByRole('row', { name: 'Laptop' })).toHaveCount(0);
```

SAY THIS: "I locate rows by role and name — getByRole row named Laptop — then chain cell locators with nth or read textContent. Counts are assertions themselves with toHaveCount, and after a delete I assert the row's count is zero, which is the web-first way: the assertion retries until the row actually disappears."

Q: How do you automate a date picker?

Understand it: Two species, one decision. If it's a real `<input type="date">` (or the widget accepts typing), `fill()` types the value in one shot — no clicking through calendars. If it's a custom widget that ONLY opens on click, you walk it by role: open it, click month arrows until the right month, then the day button.

```
// species 1: real input — one fill, done
await page.getByLabel('Start date').fill('2026-09-30');

// species 2: custom widget — walk it like a user
await page.getByLabel('Start date').click();                    // open the picker
await page.getByRole('button', { name: 'Next month' }).click();  // navigate
await page.getByRole('button', { name: /^30$/ }).click();        // the day — exact name!
```

The trap they watch for: the day button matching by exact name — `{ name: '30' }` can also match "30th of next month"; the regex `/^30$/` pins it.

Q: Canvas apps and CAPTCHA — the honest answer?

Understand it: Both are "there's no DOM to locate" questions, and the answer shows whether you know what automation IS. A canvas is a single painted image — zero elements inside. CAPTCHA is a deliberately human-only gate. Neither is "hard to automate"; both are WRONG to automate with locators.

- **Canvas:** use visual comparison (`toHaveScreenshot`) for the rendered output, or — far better — ask for test hooks/data behind the canvas.
- **CAPTCHA:** test-environment bypass or a fixed token; the login FLOW is what needs testing, the CAPTCHA is the vendor's product.

SAY THIS: "Canvas has no DOM — it's one painted element — so I verify it with image comparison or test hooks, never coordinate clicks. CAPTCHA is designed to stop me, so I don't fight it: in test environments I ask for a bypass, and I say that openly — it's a scope decision, not a skill gap."

Q: Your suite is slow and flaky. How do you triage it?

Understand it: This is a senior question — they want a METHOD, not a shrug. Slow and flaky usually have the same roots: waiting on the wrong things, shared state, and too much serial work. The trace viewer is your evidence source; the config is your lever.

The method, in order:

1. **Open the traces of the failures** — the trace shows exactly which action starved and why (element never stable? network idle never reached? assertion retried 30s?). Diagnose from evidence, not vibes.
2. **Fix the waits** — replace fixed `waitForTimeout` with web-first assertions (`await expect(...).toBeVisible()`) — auto-retry instead of blind sleeps.
3. **Check isolation** — tests sharing a logged-in state or mutating shared data WILL flake. Fresh context per test (Playwright's default) + storageState for speed, not shared contexts.
4. **Speed structurally** — `workers` up, `fullyParallel: true` where order allows, `--shard` across CI machines, API-seeded test data instead of UI-driven setup (create via API, assert via UI).
5. **Keep flakiness visible** — `retries: 1-2` as a safety net, and the report MARKS pass-on-retry as flaky — a burn-down list, not a hiding place.

SAY THIS: "I start with the trace viewer — it tells me which action starved and why, so I'm fixing evidence, not guesses. Most flakiness is fixed waits and shared state: I replace sleeps with web-first assertions, keep contexts isolated, and for speed I parallelize with workers and sharding and seed data through the API. Retries stay as a safety net and flaky-marking keeps the list honest — we burn it down, not hide it."

Q: CI is expensive. How do you run fewer browsers without losing coverage?

Understand it: The answer is a testing PYRAMID of browser coverage, not "just use Chrome". Risk concentrates in the app's own logic (any browser) and in browser ENGINE differences (Chromium vs Firefox vs WebKit). So you match the RUN to the risk.

```
# the PR gate: chromium-only smoke tag — minutes, not half-hours
npx playwright test --project=chromium --grep @smoke

# the nightly: the full matrix, everything
npx playwright test
```

```
// tag the smoke set as you write them
test('login works @smoke', async ({ page }) => { ... });
```

SAY THIS: "Pull requests run a chromium-only smoke pass — minutes, not hours. The nightly job runs the full cross-browser matrix, and WebKit/Firefox are on the nightly because engine differences are exactly what they catch. Same suite, two gates: fast feedback where developers are, deep coverage where it belongs."

SECTION 23 — CODING QUESTIONS THEY MAKE YOU WRITE LIVE (full code for all ten)

The live-round rules: they want imports thought through, fixtures used naturally, and web-first assertions instead of sleeps. The code below is the complete answer each time — read it, then retype from memory.

23.1 Login test with POM + a custom fixture (the #1 hands-on — own it cold)

What they check: page-object separation in TypeScript style, fixture fluency, and the assertion doing the waiting.

```
// ---------- pages/login.page.ts — the POM ----------
import { Page, Locator, expect } from '@playwright/test';

export class LoginPage {
    private readonly page: Page;
    private readonly username: Locator;
    private readonly password: Locator;
    private readonly submit: Locator;

    constructor(page: Page) {
        this.page = page;
        this.username = page.getByLabel('Username');
        this.password = page.getByLabel('Password');
        this.submit   = page.getByRole('button', { name: 'Sign in' });
    }

    async goto()      { await this.page.goto('/login'); }
    async login(u: string, p: string) {
        await this.username.fill(u);
        await this.password.fill(p);
        await this.submit.click();
    }
}

// ---------- tests/login.spec.ts — the test + fixture ----------
import { test as base, expect } from '@playwright/test';
import { LoginPage } from '../pages/login.page';

const test = base.extend<{ loginPage: LoginPage }>({
    loginPage: async ({ page }, use) => use(new LoginPage(page)),
});

test('valid login lands on dashboard', async ({ loginPage }) => {
    await loginPage.goto();
    await loginPage.login('admin', 'admin123');
    await expect(loginPage['page']).toHaveURL(/.*dashboard/);   // or expose a getter
});
```

Why it scores: locators live in ONE class, the fixture injects it (Playwright's answer to `new LoginPage(driver)` scattering), and the URL assertion RETRIES until navigation actually happens — no sleep.

23.2 Find a table row by name -> delete it -> assert it disappears

```
test('delete the Laptop row', async ({ page }) => {
    const row = page.getByRole('row', { name: 'Laptop' });
    await row.getByRole('button', { name: 'Delete' }).click();

    // web-first: retries until the row count hits zero — never a sleep
    await expect(page.getByRole('row', { name: 'Laptop' })).toHaveCount(0);
});
```

23.3 Mock an API to return 500 and assert the error banner

```
test('server error shows the banner', async ({ page }) => {
    await page.route('**/api/orders', route =>
        route.fulfill({ status: 500, body: JSON.stringify({ error: 'boom' }) })
    );

    await page.goto('/orders');
    await expect(page.getByRole('alert')).toContainText('Something went wrong');
});
```

The line to say while writing: "route.fulfill answers the request without the server — so this test is deterministic even when the backend is on fire."

23.4 Handle a popup with the Promise.all pattern

Understand it first: popup-opening methods fire the navigation and return BEFORE the popup exists — so you must START listening first, then click. `Promise.all` overlaps the two.

```
const [popup] = await Promise.all([
    page.waitForEvent('popup'),                        // arm the listener FIRST
    page.getByRole('link', { name: 'Terms' }).click(), // then act
]);
await expect(popup).toHaveTitle(/Terms/);   // a popup is just another Page
await popup.close();
```

The follow-up: "How is a tab different from a window?" -> nothing, both are Page objects — same API, no switchTo.

23.5 Reach into an iframe with frameLocator

```
const editor = page.frameLocator('#editor-frame');   // scope — no switchTo, no exit

await editor.getByRole('textbox').fill('hello iframe');
await expect(editor.getByRole('status')).toContainText('Saved');
// no switch back — the MAIN page locator was never interrupted
```

The line to say: "frameLocator is a scoped view, not a mode — my main-page locators keep working the whole time."

23.6 storageState — log in ONCE for the whole suite (the setup project)

```
// auth.setup.ts — a project that ONLY produces the saved session
import { test as setup } from '@playwright/test';

setup('authenticate', async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Username').fill('admin');
    await page.getByLabel('Password').fill('admin123');
    await page.getByRole('button', { name: 'Sign in' }).click();
    await page.waitForURL('**/dashboard');

    await page.context().storageState({ path: 'auth/state.json' });  // THE save
});
```

```
// playwright.config.ts — every dependent project reuses it
export default defineConfig({
    use: { storageState: 'auth/state.json' },   // tests START logged in
    projects: [
        { name: 'setup', testMatch: /auth.setup.ts/ },
        { name: 'e2e', dependencies: ['setup'] },   // setup runs first
    ],
});
```

23.7 Data-driven login from JSON — a loop with dynamic test titles

```
import { test, expect } from '@playwright/test';
import users from '../data/users.json';          // [{user, pass, expected}]

for (const { user, pass, expected } of users) {
    test(`login as ${user} expects ${expected}`, async ({ page }) => {
        await page.goto('/login');
        await page.getByLabel('Username').fill(user);
        await page.getByLabel('Password').fill(pass);
        await page.getByRole('button', { name: 'Sign in' }).click();

        await expect(page.getByRole('alert'))
            .toContainText(expected);
    });
}
```

The gotcha they probe: the test must be CREATED inside the loop (dynamic title per row) — defining one test that loops internally reports one result and fails all rows together.

23.8 Upload a file and verify the success toast

```
import path from 'path';

test('upload receipt', async ({ page }) => {
    await page.setInputFiles('input[type=file]',
        path.resolve(__dirname, '../data/receipt.pdf'));   // no dialog, no OS popups

    await expect(page.getByRole('status')).toContainText('Upload complete');
    // no file chooser dance: setInputFiles points straight at the input
});
```

23.9 expect.soft — collect ALL form validation messages

```
test('empty form shows every message', async ({ page }) => {
    await page.goto('/signup');
    await page.getByRole('button', { name: 'Register' }).click();  // submit empty

    await expect.soft(page.getByText('Username is required')).toBeVisible();
    await expect.soft(page.getByText('Email is required')).toBeVisible();
    await expect.soft(page.getByText('Password is required')).toBeVisible();
    // soft = record and continue — the report lists ALL failures, not just the first
});
```

The contrast line: "expect.soft is Playwright's SoftAssert — same collect-and-report philosophy, same assertAll idea, built into the assertion itself."

23.10 Visual screenshot of the navbar (image as assertion)

```
test('navbar looks right', async ({ page }) => {
    await page.goto('/');
    await expect(page.getByRole('navigation')).toHaveScreenshot('navbar.png');
    // first run: npx playwright test --update-snapshots  (creates the baseline)
});
```

The follow-up: "What if it fails?" -> the report shows actual vs expected vs DIFF, pixel by pixel — you either fix the UI or consciously update the baseline.

SECTION 24 — RAPID-FIRE, FULLY EXPLAINED (final revision — every line now with its why)

How to use this section: the first line is the one-breath answer; the paragraph under it is what you say when they say "explain more".

- **Playwright speaks WebSocket, Selenium speaks HTTP.**

  Selenium sends each command as a separate HTTP request — ask, wait for reply, ask again. Playwright opens ONE persistent WebSocket to the browser server and streams commands both ways — no round-trip per click. That's a big part of why Playwright is faster, and it's the one-liner interviewers use to check you know there IS an architecture difference.

- **Browser -> context -> page; context = incognito session; fresh context per test = isolation.**

  The browser is the process; a context is an isolated incognito-style session inside it (own cookies, own storage); pages are tabs inside the context. The framework gives every test a FRESH context, so no test can smell another test's cookies — that isolation is Playwright's #1 flake-killer.

- **Auto-waiting checks visible + stable + receives events + enabled — before every action.**

  Four checks, on every click/fill/hover: the element is rendered, it has STOPPED MOVING (not animating), nothing is covering it, and it's not disabled. Auto-waiting is why Playwright tests rarely need explicit waits — the wait IS the action.

- **Locators: getByRole first, getByTestId when nothing semantic survives, CSS/XPath last.**

  getByRole binds to the ACCESSIBILITY tree — what the element IS to a screen reader — so it survives redesigns. getByTestId is the deliberate stable hook. CSS is implementation-coupled but fine, XPath is the last resort: powerful, slow, brittle to refactor.

- **Strict mode: multiple matches = instant failure; disambiguate with first/nth/filter.**

  `page.locator('button')` matching 5 buttons fails LOUDLY instead of quietly clicking a random one — silent ambiguity is where bugs hide. Fix ambiguity with `.first()`, `.nth(1)`, `.filter({ hasText })`, or a better locator — never `.first()` as a reflex (that's how you test the wrong element forever).

- **expect() retries; .isVisible() answers once.**

  `expect(locator).toBeVisible()` re-checks until timeout — it WAITS for truth. `await locator.isVisible()` is a snapshot: true/false right now, zero retry, zero failure. Rule: assertions you want to pass eventually = expect(); questions about the current instant (e.g. "is the spinner gone?") = the method.

- **Fixtures are injected services: page, context, browser, request, browserName.**

  Instead of every test creating/managing its own browser, fixtures HAND you ready objects — the framework sets up, tears down, and can be EXTENDED with your own (that's how the LoginPage fixture and userData fixtures work — same injection machinery).

- **beforeEach = @BeforeMethod, beforeAll = @BeforeClass, no beforeSuite — config is the suite.**

  Hooks map one-to-one for your TestNG memory. There's no beforeSuite because in Playwright "once for everything" isn't a hook — it's the CONFIG and projects (and setup projects for things like authentication). The suite level is declarative, not code.

- **projects = cross-browser matrix; workers = parallel processes; sharding = split across CI machines.**

  Three different axes of "more": projects run the same tests in different browsers/devices (the WHAT), workers run test FILES in parallel OS processes (the HOW FAST), sharding splits the suite across CI machines (the HOW MANY MACHINES — no Grid, no nodes, one flag).

- **storageState = login once, tests inherit the session.**

  One setup project does the real login and saves cookies + localStorage to state.json; every test then starts with the session injected — authentication, once, for the whole suite. It's Selenium's cookie-reuse idea, first-class and automatic.

- **route.fulfill = mock; route.abort = block; request fixture = API testing.**

  The three network moves: fulfill answers a request with your own body/status (mock the backend), abort kills it (test offline/failure paths), and the `request` fixture fires real API calls without any browser at all — API testing and UI testing in one tool.

- **Trace viewer = flight recorder of the run.**

  On failure, trace.zip records every action with the DOM snapshot, network, console and timing — open it with `npx playwright show-trace trace.zip` and you can step to the exact millisecond and SEE the page. It's a black box recorder for tests, and saying "it's how I debug in CI without a VNC" lands very well.

- **frameLocator = iframe scope without switching; Promise.all = listen + click.**

  frameLocator gives a scoped VIEW of the iframe — main-page locators never stop working (no switchTo, nothing to switch back from). Promise.all is the timing pattern for events you must ARM before triggering: start waitForEvent and the click together, so the popup can't fire before you're listening.

- **Tabs are Page objects — no switchTo; bringToFront() is only for eyes; context.pages() lists them all.**

  Every tab is just another Page with the same locator/action API — no window-handle juggling or Set loops. bringToFront() only matters for human WATCHING (and screenshots); the driver can act on a background tab fine. context.pages() is your getWindowHandles equivalent.

- **Retries rerun failures; pass-on-retry = marked flaky.**

  `retries: 2` reruns a failed test up to twice — and when it passes on a retry, the report marks it FLAKY (yellow), not green. That's the design: retries keep the pipeline moving, flaky-marking keeps you honest enough to burn the cause down.

- **Upload = setInputFiles; download = waitForEvent + saveAs.**

  Upload points straight at the hidden `<input type=file>` — no OS dialog ever appears, no Robot class. Download is the arm-then-act pattern: waitForEvent('download') and the click together, then download.saveAs(). Headless-safe, CI-safe, zero OS involvement.

SECTION 25 — CLOSING ANSWER + SELENIUM-TO-PLAYWRIGHT TRANSLATION

Q: "Why should we hire you for this Playwright role when your background is Selenium?"

> *"The testing concepts transfer 1:1 — Page Objects, data-driven design, isolation, reporting, CI discipline. I just switch dialects: explicit waits become auto-waiting, findElement becomes getByRole, ITestListener screenshots become a config line, testng.xml groups become grep tags, and the Grid becomes workers + sharding. I understand WHY each piece exists, so I pick up the how quickly — and I can talk to both stacks, which helps when the team still maintains Selenium suites."*

The Selenium -> Playwright translation table (memorize — you'll be asked at least 3 rows):

Selenium world	Playwright world
`driver.findElement(By.id("x"))`	`page.getByTestId('x')` / `getByRole(...)`
Explicit wait + ExpectedConditions	auto-waiting (built-in)
`Assert.assertEquals`	`await expect(...).toHaveText(...)` (web-first, retries)
`SoftAssert`	`expect.soft(...)`
`@BeforeMethod / @AfterMethod`	`beforeEach / afterEach`
`@BeforeClass / @AfterClass`	`beforeAll / afterAll`
testng.xml (suite, groups, parallel)	playwright.config.ts (projects, grep, workers)
DataProvider + Excel/POI	JSON/CSV loop + fixtures
`switchTo().frame()`	`frameLocator()`
window handles / `getWindowHandles`	`waitForEvent('popup')` + Promise.all
ITestListener screenshot on failure	`use: { screenshot: 'only-on-failure' }`
Extent Reports	built-in HTML reporter + trace viewer
Selenium Grid	workers + `--shard`
`switchTo().alert()`	`page.on('dialog')` handler
Maven `pom.xml`	`package.json` + config
`mvn clean test`	`npx playwright test`
Java + TestNG + Extent glue	one `npm i -D @playwright/test`

Last-day revision plan

1. Say the 60-second framework script 5 times (Section 3).
2. Recite "Config Runs Projects, Pages Keep Fixtures, Tests Expect Traces" and sketch the project tree.
3. Recite the auto-wait actionability checklist (visible, stable, receives events, enabled).
4. Recite the flow: Read -> Discover -> Schedule -> Isolate -> Assert -> Retry -> Report.
5. Read Section 24 rapid-fire twice the night before.

Walk in knowing BOTH tools — Selenium for depth, Playwright for the modern edge. That combination is exactly what teams are hiring.

SECTION 26 — THE REMAINING CORNER (extras interviewers check)

26.1 globalSetup / globalTeardown ("how do you do something ONCE before ALL tests?")

Understand it: Hooks run per test (beforeEach) or per file (beforeAll). For once-per-RUN work (seed data in a DB, start a test server, generate a token), the config points to a global setup file:

```
// playwright.config.ts
export default defineConfig({
  globalSetup: './global-setup.ts',       // runs once before everything
  globalTeardown: './global-teardown.ts', // runs once after everything
});
```

Modern alternative for auth: a **setup project with dependencies** (Section 12) — prefer it for login, use globalSetup for environment-level work.

MEMORY HOOK: *"beforeEach = per test, beforeAll = per file, globalSetup = per RUN."*

26.2 test.step(), test.info() and test.attach() (report polish questions)

```
await test.step('Submit the payment form', async () => {   // shows as a grouped step in the HTML report
  await page.getByRole('button', { name: 'Pay' }).click();
});

test.info().annotations.push({ type: 'issue', description: 'JIRA-1234' });  // tag the run
await test.attach('failure-screenshot', { path: 'shot.png', contentType: 'image/png' });  // attach evidence
```

SAY THIS: "We wrap logical phases in test.step so the report reads like the test plan, and attach evidence with test.attach — the equivalent of Extent's log-and-screenshot, but first-party."

26.3 expect.poll() and toPass() (advanced assertion questions)

- **expect.poll** — retry a NON-locator value until it satisfies the assertion (locator assertions can't see it — e.g., a number read from the UI or API):

```
await expect.poll(async () => getCartCountFromAPI()).toBeGreaterThan(3);
```

- **toPass()** — retry an arbitrary async block until it stops throwing:

```
await expect(async () => {
  await page.getByRole('button').click({ trial: true });
}).toPass({ timeout: 10_000 });
```

MEMORY HOOK: *"poll for values, toPass for attempts, expect(locator) for elements."*

26.4 addInitScript — run JS BEFORE the page's own scripts

Understand it: some state must exist before the app boots (localStorage, feature flags, mocked date). `addInitScript` injects your script into every page before anything else runs:

```
await page.addInitScript(() => {
  localStorage.setItem('feature-flag', 'new-checkout');
});
await page.goto('/');   // app reads the flag already set
```

MEMORY HOOK: *"storageState = saved session; addInitScript = state that never even renders."*

26.5 Component testing (know the concept)

Playwright can test a SINGLE React/Vue/Svelte component in isolation — it mounts just that component (no full app, no router) and asserts on it. Faster than E2E for UI logic; E2E still covers the journeys. Say: "We keep E2E for critical flows and use component tests where a full browser run is overkill."

26.6 Touch actions (mobile emulation)

On a mobile device emulation, `page.tap(selector)` is the touch-tap (like a click without hover semantics); swipes are done with touch action points (mouse down/move/up). Know `tap()` by name for "how do you test mobile gestures?"

26.7 Playwright in Java / Python / .NET

Same engine, same concepts (locators, auto-wait, contexts) — but the **richest experience is TypeScript/JavaScript**: fixtures, UI mode and new features land there first. Java uses JUnit/TestNG integration, Python uses pytest. Say: "Concepts are identical across bindings; I work in TypeScript because that's where the ecosystem leads."

26.8 BDD with Playwright

Gherkin works here too: **playwright-bdd** (or cucumber-js) maps Given/When/Then steps onto Playwright code — the Selenium guide's Section 18.8 Cucumber knowledge transfers directly. Say: "Same Gherkin on top, Playwright driving underneath — the step definitions call the same page objects."

SECTION 27 — ASSIGNMENTS (the live-round coding bank — Q15 to Q34)

These are the exact hands-on assignments interviewers hand to candidates. For each: **Candidate Answer** (the code to write, with the reasoning that earns marks) and the **Interviewer Follow-up** you should expect right after. (Assignments 1–14 of the classic list — locators, waits, screenshots, dropdowns, tabs, dialogs, drag-and-drop, hover, table reading, assertions — are all fully coded earlier in this guide.)

PART A — CODING ASSIGNMENTS

Q15. Upload a file and verify the uploaded file name. [CODING]

Candidate Answer:

```
import { test, expect } from '@playwright/test';

test('upload invoice and verify its name', async ({ page }) => {
  await page.goto('/upload');

  const fileName = 'invoice-2026.pdf';

  // Point DIRECTLY at the <input type="file"> — no dialog, no OS popups, headless-safe
  await page.setInputFiles('input[type="file"]', `test-data/${fileName}`);

  // Verify the UI confirms the upload with the exact file name
  await expect(page.getByTestId('uploaded-name')).toHaveText(fileName);

  // Stronger: verify it appears in the file list as a row
  await expect(page.getByRole('row', { name: fileName })).toBeVisible();
});
```

Why this answer scores: setInputFiles bypasses the OS dialog entirely (the classic Selenium Robot-class problem does not exist); the verification is web-first — `toHaveText` retries until the async upload finishes, so no manual sleep after upload. Mentioning the row-check shows you verify the SERVER accepted it, not just the UI.

Interviewer Follow-up: *"What if the uploader is a drag-and-drop zone with no `<input>` tag?"*

> *"Then I listen for the file chooser event:* `const fc = page.waitForEvent('filechooser'); await dropzone.click(); (await fc).setFiles('path');` *— Playwright intercepts the browser's file picker programmatically."*

Q16. Handle an iframe and interact with an element inside it. [CODING]

Candidate Answer:

```
test('type inside the editor iframe', async ({ page }) => {
  await page.goto('/editor');

  const editor = page.frameLocator('#editor-frame');   // enter the frame's world

  await editor.getByLabel('Comment').fill('Hello from the frame');
  await editor.getByRole('button', { name: 'Post' }).click();

  // nested frames chain naturally:
  // const inner = page.frameLocator('#outer').frameLocator('#inner');
});
```

Why this answer scores: `frameLocator` scopes EVERY locator inside the frame — there is no switchTo/frame-exit dance like Selenium (`switchTo().frame()` + `defaultContent()`). The frame reference is declarative, auto-waited, and reusable. Also know: `page.frames()` lists all frames (including the main frame) — handy for debugging "where is my element?".

Interviewer Follow-up: *"How do you even know the element is inside an iframe?"*

> *"In DevTools the element sits under an* `<iframe>` *ancestor, and* `page.frames()` *shows the frame tree. In the trace viewer you can see the frame each action hit."*

Q17. Write a test using getByRole() and explain why it is preferable to brittle XPath. [CODING]

Candidate Answer:

```
test('sign in via role-based locators', async ({ page }) => {
  await page.goto('/login');

  // BRITTLE alternatives (what we avoid):
  //   await page.locator('/html/body/div[2]/div[1]/form/div[3]/button[1]').click();  // absolute path
  //   await page.locator('//div[@class="btn btn-primary mt-2"]').click();             // styling class

  // ROBUST — what a user and a screen reader perceive:
  await page.getByLabel('Email').fill('qa@example.com');
  await page.getByLabel('Password').fill('Secret@123');
  await page.getByRole('button', { name: 'Sign in' }).click();

  await expect(page.getByRole('heading', { level: 1 })).toContainText('Welcome');
});
```

Why it is preferable (the explanation they want): the role + accessible name targets **what the user experiences**, not how the developer built it. An absolute XPath breaks if ANY tag in the chain is added, removed or reordered; a class-based XPath breaks when the designer changes `mt-2` to `mt-3`. `getByRole('button', { name: 'Sign in' })` survives markup refactors (div -> button), CSS redesigns and DOM reordering — and as a bonus it tests the accessible tree, so failures expose real accessibility bugs (a div with an onClick and no role will NOT match `getByRole('button')` — which is a genuine product defect). Strict mode then gives a clear error if two buttons match, instead of silently clicking the first one.

Interviewer Follow-up: *"When would you legitimately fall back to CSS or XPath?"*

> *"When nothing semantic is stable — I'd first ask the team to add a `data-testid` and use `getByTestId`. CSS for ids and data-attributes. XPath almost never — it can't be matched against the accessible tree and is the slowest, most brittle option."*

Q18. Use test.beforeEach() correctly and explain setup versus test logic. [CODING]

Candidate Answer:

```
test.describe('Cart', () => {

  test.beforeEach(async ({ page }) => {                  // SETUP — identical for every test
    await page.goto('/products');
    await page.getByRole('button', { name: 'Add to cart' }).first().click();
    await page.getByRole('link', { name: 'Cart' }).click();
  });

  test('shows one item', async ({ page }) => {           // TEST LOGIC — only the unique part
    await expect(page.getByRole('listitem')).toHaveCount(1);
  });

  test('can remove the item', async ({ page }) => {
    await page.getByRole('button', { name: 'Remove' }).click();
    await expect(page.getByTestId('empty-cart')).toBeVisible();
  });
});
```

The rule (say this): setup = everything IDENTICAL across tests — the Arrange. Test body = only the unique Act + Assert. If a block of steps appears in more than one test, it belongs in beforeEach, in a fixture, or in a page-object method — not copy-pasted. And `beforeAll` is per WORKER with no shared context, so login-in-beforeAll doesn't work — use beforeEach or storageState for sessions.

Interviewer Follow-up: *"When would you NOT put login in beforeEach?"*

> *"When tests need different roles — then a setup project with storageState per role is better — or when only a subset needs it, I scope it with a nested describe and its own beforeEach."*

Q19. Create a parameterized test using an array of test data. [CODING]

Candidate Answer:

```
const credentials = [
  { role: 'admin',    pass: 'Admin@123', expected: 'Admin Dashboard' },
  { role: 'shopper',  pass: 'Shop@123',  expected: 'My Account' },
  { role: 'locked',   pass: 'wrong',     expected: 'Invalid credentials' },
];

for (const c of credentials) {
  test(`login as ${c.role} shows "${c.expected}"`, async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Username').fill(c.role);
    await page.getByLabel('Password').fill(c.pass);
    await page.getByRole('button', { name: 'Sign in' }).click();
    await expect(page.getByRole('heading', { level: 1 })).toContainText(c.expected);
  });
}
```

Why this is the right shape: the test is CREATED inside the loop at file top level — 3 rows become 3 independent tests in the report, each with a readable dynamic title and its own isolated context. This is the TypeScript equivalent of TestNG's DataProvider, but with zero annotations — just a loop. (The classic mistake is putting the loop INSIDE one `test()` — that gives one test, one report line, and one failure killing all rows.)

Interviewer Follow-up: *"How would you feed this from Excel/JSON instead?"*

> *"Same pattern — read the file once (JSON import or a CSV/Excel reader util) and loop over the parsed rows; the data source changes, the loop structure stays."*

Q20. Use storageState for an authenticated session. [CODING]

Candidate Answer:

```
// ===== 1. tests/auth.setup.ts — logs in ONCE, saves the session =====
import { test as setup } from '@playwright/test';

setup('authenticate', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('qa@example.com');
  await page.getByLabel('Password').fill('Secret@123');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await page.waitForURL('/dashboard');
  await page.context().storageState({ path: '.auth/user.json' });
});

// ===== 2. playwright.config.ts — run setup first, tests inherit the session =====
export default defineConfig({
  projects: [
    { name: 'setup', testMatch: /auth\.setup\.ts/ },
    { name: 'chromium',
      dependencies: ['setup'],                          // ordering guarantee
      use: { storageState: '.auth/user.json' } },        // every test starts LOGGED IN
  ],
});
```

The explanation to give: login happens once per RUN (not per test), and every test still gets a FRESH isolated context — it simply starts with the saved cookies and localStorage injected. That's speed (skip 200 UI logins) plus isolation (no shared session state). Multi-role setups save one file per role (`.auth/admin.json`, `.auth/customer.json`) and point different projects at them.

Interviewer Follow-up: *"What's inside the .json, and when does it break?"*

> *"Cookies and origin-bound localStorage for each domain. It breaks when the session expires — then I re-run the setup project; and the `.auth/` folder is gitignored because it's a credential artifact."*

HOW WE USED IT — the complete project walkthrough (say this when asked "how did you use storageState in YOUR project?"):

Step by step, exactly as it ran in our suite:

1. **Created the setup spec** (`tests/auth.setup.ts`) that performs the UI login ONCE and calls `page.context().storageState({ path: '.auth/user.json' })` — which snapshots all cookies + localStorage of the logged-in session into that JSON file.

2. **Wired it in the config** as a separate project with `testMatch: /auth\.setup\.ts/`, and made the chromium project **depend on it** (`dependencies: ['setup']`) — so the runner guarantees: setup finishes BEFORE any real test starts, in every run, in every CI shard, with zero Jenkins scripting.
3. **Every test consumes it transparently** — because `use: { storageState: '.auth/user.json' }` sits on the project, the built-in `page` fixture loads that JSON while creating each test's BrowserContext and injects the cookies BEFORE any page opens:

```
// tests/dashboard.spec.ts — NOTE: no login steps anywhere
import { test, expect } from '@playwright/test';

test('dashboard greets the saved user', async ({ page }) => {
  await page.goto('/dashboard');     // arrives ALREADY authenticated — cookie was injected
  await expect(page.getByRole('heading', { name: 'Welcome' })).toBeVisible();
  // the 8-step login flow (~4 seconds) never ran — for this test or any other
});
```

4. **Isolation is preserved** — each test gets its OWN private copy of the session (a fresh context with the same cookies injected). Two tests mutating "their" session cannot leak cookies into each other — speed of a shared session, safety of isolated ones.
5. **Roles** — we saved two states (`.auth/admin.json` from an admin login, `.auth/customer.json` from a customer login) and defined two projects, each with its own storageState — admin tests and customer tests both start where they need to be.
6. **Housekeeping we actually did:** `.auth/` in `.gitignore` (credential artifact — never committed); when the session expired, tests suddenly landed on `/login` instead of the dashboard — the tell-tale symptom of a STALE storageState — fixed by re-running the setup project; and for one performance-critical suite we skipped the browser entirely: logged in via `request.post('/api/login')`, built the storageState JSON from the returned token, and injected that — no UI login in the whole pipeline.

The mistakes we explicitly avoided (a favorite follow-up):

Mistake	Symptom	Correct way
Login steps still left inside tests	slow suite, storageState pointless	remove login from ALL specs — skipping it was the entire point
Calling storageState() in beforeAll	doesn't work as expected — beforeAll has no page/context per test	setup PROJECT with dependencies, not hooks
Wrong path in config / setup never ran	tests land on /login	verify path + that `dependencies: ['setup']` is set
Committing .auth/ to Git	credential leak in the repo	.gitignore + regenerate per run
Reusing one state for a role-mix app	admin/customer tests interfere	one storageState file PER role, one project per role

MEMORY HOOK: *"Setup saves it, config wires it, tests inherit it — login ONCE per run, isolated EVERY run."*

Q21. Write an API test using request.get() and validate status and response body. [CODING]

Candidate Answer:

```
import { test, expect } from '@playwright/test';

test('GET /users/1 returns the right user', async ({ request }) => {
  const res = await request.get('/api/users/1');        // baseURL from config applies

  // 1. status code
  expect(res.status()).toBe(200);

  // 2. response body — specific fields and shape
  const body = await res.json();
  expect(body.id).toBe(1);
  expect(body.email).toMatch(/.+@.+\..+/);
  expect(body).toMatchObject({ id: 1, name: expect.any(String) });
});
```

```
test('POST /orders creates an order', async ({ request }) => {
  const res = await request.post('/api/orders', {
    data: { item: 'laptop', qty: 1 },
    headers: { Authorization: `Bearer ${process.env.API_TOKEN}` },
  });
  expect(res.status()).toBe(201);
  expect((await res.json()).status).toBe('created');
});
```

Why this scores: the `request` fixture is a built-in API client — no RestAssured, no extra dependency; the assertions show both exact-value and shape (`toMatchObject` with `expect.any`) validation, and the POST shows auth headers from env vars (never hardcode tokens).

Interviewer Follow-up: *"Why keep API tests inside the UI framework?"*

> *"Two reasons: speed — API smoke tests catch breakage in seconds — and setup/cleanup — we create test data via API, run the UI flow on it, then clean up via API, instead of slow UI forms."*

Q22. Generate unique test data using timestamp or UUID. [CODING]

Candidate Answer:

```
// Timestamp — human-readable, sortable
const email = `qa_${Date.now()}@testmail.dev`;

// UUID — collision-proof (128-bit random)
const ref    = crypto.randomUUID();                    // built into Node, no import
const email2 = `qa_${crypto.randomUUID().slice(0, 8)}@testmail.dev`;

test(`register ${email} and verify welcome`, async ({ page }) => {
  await page.goto('/register');
  await page.getByLabel('Email').fill(email);
  await page.getByLabel('Full name').fill(`QA ${ref}`);
  await page.getByRole('button', { name: 'Create account' }).click();

  await expect(page.getByRole('heading')).toContainText('Welcome');
  await expect(page.getByTestId('user-email')).toHaveText(email);
});
```

The reasoning to state: parallel workers mutating the same account is the #1 source of random CI failures — unique data per run removes the collision AND makes re-runs safe (yesterday's leftover user no longer blocks today's test). Prefer UUID over `Date.now()` when many tests start in the same millisecond (workers can genuinely collide on a timestamp; they cannot on a UUID). With faker (Section 21.1) you get realistic names/emails on top of uniqueness.

Interviewer Follow-up: *"How do you clean this data up?"*

> *"Best-effort API delete in afterEach using the unique id we captured, plus a nightly cleanup job. The unique-name pattern makes cleanup optional rather than load-bearing — a failed delete doesn't break the next run."*

Q23. Configure Chromium, Firefox, and WebKit with retries only in CI. [CODING]

Candidate Answer:

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  retries: process.env.CI ? 2 : 0,        // <- the whole trick: env check
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox',  use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit',   use: { ...devices['Desktop Safari'] } },
  ],
});
```

Explain it: Jenkins and GitHub Actions export `CI=true` automatically, so the same config retries twice in CI and zero times locally — locally you WANT to see the failure immediately (retries would hide the flakiness you're supposed to fix); in CI a retry absorbs transient environment noise, and a test that passes on retry is marked `flaky` in the report so it isn't silently forgiven. Three projects = every spec runs on all three engines; one spec file

× 3 projects = 3 test runs.

Interviewer Follow-up: *"How do you run just one browser, or split the suite across CI machines?"*

> *"`npx playwright test --project=chromium` for one browser, and sharding — `--shard=1/4` — to split the suite across machines, with each shard running its own workers."*

Q24. Configure base URL, timeout, screenshot, video, trace, and HTML reporter. [CODING]

Candidate Answer:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  timeout: 30_000,                        // per-test limit
  use: {
    baseURL: 'https://staging.myapp.com',// page.goto('/login') just works
    actionTimeout: 15_000,              // per-action default wait
    screenshot: 'only-on-failure',       // evidence, but only when needed
    video: 'retain-on-failure',
    trace: 'retain-on-failure',
  },
  reporter: [
    ['html', { open: 'never' }],         // playwright-report/index.html
    ['list'],                            // console progress
  ],
});
```

Explain each: `baseURL` turns relative paths into full URLs (switch environments with one line); `timeout` bounds the whole test, `actionTimeout` each action; the `'only-on-failure'` / `'retain-on-failure'` values mean evidence is captured ALWAYS but kept only for failures — full video on every test would fill the disk; `reporter` array runs both the HTML report (for humans) and the list reporter (for console/CI logs).

Interviewer Follow-up: *"Where do the artifacts go, and how do you view a trace?"*

> *"Into `test-results/<test-name>/` — trace.zip, video.webm, screenshots. `npx playwright show-trace trace.zip` opens the trace viewer with DOM snapshots, network and console per step."*

PART B — DEBUGGING EXERCISES

Q25. A locator times out even though the element is visible. How do you debug it? [CODING]

Candidate Answer — recite it as a checklist:

1. **Read the error first** — Playwright prints the exact locator, its options, and what it waited for; the message usually names the real problem (strict mode violation vs timeout).
2. **Strict-mode check** — does the locator resolve to MULTIPLE elements? Then Playwright fails ON PURPOSE instead of clicking the first. Fix with `.first()`, `.nth()`, or `.filter({ hasText })`.
3. **Iframe check** — an element inside an `<iframe>` is invisible to page locators. Fix: `page.frameLocator(...)`. (`page.frames()` lists the frames; DevTools shows the iframe ancestor.)
4. **Actionability check** — "visible to my eyes" is not "clickable to Playwright": an invisible overlay, a spinner, an ongoing CSS animation, or a disabled state blocks the action even though you can SEE the button. Fix: wait for the overlay to disappear (`expect(overlay).toBeHidden()`), or confirm the diagnosis with `click({ force: true })` — diagnostic only, not a permanent fix.
5. **SPA/tab check** — did the action open a new tab or navigate without reload? Capture the popup or wait for the URL (`waitForURL`).
6. **Tools:** run with `--debug` (inspector shows the locator resolving live), open the trace, or use UI mode's time travel to see exactly what the locator matched at each step.

Interviewer Follow-up: *"What's the FIRST thing you check?"*

> *"The error message — it distinguishes strict-mode (multiple matches), not-found, and not-actionable cases, and each has a completely different fix."*

Q26. A test passes headed but fails headless in CI. What do you check? [CODING]

Candidate Answer — ordered by likelihood:

1. **Race conditions hidden by slowness** — headed mode is slower, giving the app time to settle; headless runs faster and exposes missing awaits / missing web-first assertions. This is the #1 cause.
2. **Animations/transitions** — elements mid-animation fail the "stable" actionability check differently headless. Fix: `use: { reducedMotion: 'reduce' }`.
3. **Viewport** — headless defaults to 1280×720; a responsive breakpoint may collapse your element into a hamburger. Fix: set the viewport in config.
4. **Environment, not rendering** — missing env vars/secrets in CI, different test data, timezone/locale. Fix: pin `locale`, `timezoneId`, and check the CI env.
5. **Bot detection / different user agent** — some apps serve different markup headless. Fix: `channel: 'chrome'` or headed-in-CI via `xvfb-run`.
6. **Resource limits** — too many workers on a small CI box causing browser crashes. Fix: lower `workers`.

Method, not guesswork: enable `trace: 'on'`, `video: 'on'` for the failing test in CI, read the trace exactly like a local failure.

Interviewer Follow-up: *"How do you reproduce a CI-only failure locally?"*

> *"Run the identical command (`npx playwright test`) locally first, then replicate the CI factors one by one — headless, viewport, env vars — until one flips the result; that, plus the trace, pinpoints it."*

Q27. Parallel tests modify the same account and randomly fail. How do you isolate data? [CODING]

Candidate Answer:

First, frame the root cause correctly: **the browser side is already isolated** (fresh context per test — no cookie clashes). The race is in the BACKEND data — two workers doing read-modify-write on the same account row. So the fix is data isolation, not more waits:

1. **Unique data per test** — timestamp/UUID account names (Q22): every run creates its own user, so no test can touch another's data.
2. **A user pool per role/worker** — worker 1 uses qa_user_1, worker 2 qa_user_2 (workers have an index available via `testInfo.workerIndex`).
3. **Shard-level separation** — each CI shard gets its own data slice.
4. **Move shared-resource operations to API tests** — if the flow only manipulates data (no UI value), a sequential API test is more reliable than a parallel UI one.
5. **Last resort:** a mutex/lock around genuinely-shared fixtures — but treat it as a design smell.

How to prove which test interfered: run the suite with `--workers=1` (passes?) vs full parallel (fails?) — that alone confirms a parallelism bug; then bisect with `--grep`.

Interviewer Follow-up: *"Why not just add retries?"*

> *"Retries hide the symptom — the tests still race, you just rerun the lottery. Retries are for genuine environment flake; shared-data races need isolation."*

Q28. Explain likely causes of 'Target page, context or browser has been closed'. [CODING]

Candidate Answer — the likely causes, in order:

1. **Your own code closed it** — an early `page.close()` / `context.close()` / `browser.close()` while later steps still reference the object (a classic in `beforeAll` misuse or cleanup helpers).
2. **The test finished but async work continued** — an awaited operation racing after teardown (e.g., an event handler or a fire-and-forget call using the page after the test ended).
3. **The browser crashed** — out-of-memory in CI (too many workers, Docker memory caps, video+trace on for every test).
4. **Fixture/test-structure errors** — storing a `page` from one test in a shared variable and using it in another (each test's context dies with the test); or a fixture override that closes things prematurely.

5. **Hard timeouts** in hooks killing the context mid-flow.

Debug: the stack trace points at the line using the dead object — then work BACKWARDS to find who closed it; enable traces and check the timeline's last event; reduce `workers` if it's a crash (memory).

Interviewer Follow-up: *"How do you prevent the shared-page version?"*

> *"Never store `page` outside the test that owns it — pages are per-test fixtures. Anything cross-test goes through storageState or a fresh page from a custom fixture."*

Q29. Refactor a suite full of waitForTimeout() calls using auto-waiting and web-first assertions. [CODING]

Candidate Answer:

```
// ================= BEFORE — blind, slow, flaky =================
await page.goto('/search');
await page.fill('#q', 'laptop');
await page.click('#search');
await page.waitForTimeout(5000);                    // <- hoping 5s is enough
await page.click('#results li:first-child');
await page.waitForTimeout(2000);                    // <- hoping again
await expect(page.locator('.title')).toBeVisible(); // may still race

// ================= AFTER — deterministic, faster, stable =================
await page.goto('/search');
await page.getByPlaceholder('Search').fill('laptop');
await page.getByRole('button', { name: 'Search' }).click();

// 1. auto-waiting covers the click — element must be ready before it fires
// 2. web-first assertions poll until TRUE — no fixed seconds:
const results = page.getByRole('listitem');
await expect(results.first()).toBeVisible();
await results.filter({ hasText: 'Laptop' }).first().click();

// 3. for things actions cannot see, wait for the SPECIFIC trigger:
await page.waitForResponse(r => r.url().includes('/api/search') && r.status() === 200);
await page.waitForURL('**/results**');              // SPA navigations
await expect(page.getByRole('heading', { name: 'Laptops' })).toBeVisible();
```

The refactor rule: every `waitForTimeout(n)` becomes ONE of three things — usually NOTHING (auto-waiting already covers it), otherwise a web-first `expect(...)`, otherwise a targeted `waitFor...` (URL / response / hidden). Result: the suite gets faster (no fixed sleeps) and more stable (no guessed durations).

Interviewer Follow-up: *"Is waitForTimeout EVER acceptable?"*

> *"Only while debugging interactively. Our lint/review rule bans it in committed tests — a sleep is a bet that the machine is slow enough."*

PART C — INTERVIEW FOLLOW-UPS (the meta questions)

Q30. For each solution, explain time and space complexity. [FOLLOW-UP]

Candidate Answer:

Frame it honestly for test code: browser automation is I/O-bound, so complexity lives in the DATA loops, not the awaits:

- Reading/scanning a table of n rows is **O(n)** time, **O(1)** extra space (stream the rows, don't materialize arrays you don't need).
- Filtering inside a per-row loop is **O(n²)** — refactor to one pass with a Map/Set (**O(n) time, O(n) space**): e.g., building a `Map<rowText, row>` once, then lookups are O(1).
- String concatenation in a loop is O(n²) with `+=` — collect in an array and `join` (**O(n)**).
- For pure algorithmic answers (they sometimes pivot): sorting is O(n log n); dedup via Set is O(n)/O(n); a nested-scan search is O(n·m) vs a hash-map lookup O(1) after O(n) build.

SAY THIS: "Test code is I/O-bound, so I optimize data loops: single linear passes, Set/Map for lookups, generators for huge datasets — and I avoid asserting over enormous DOMs when an API check proves the same thing faster."

Q31. How would you make the solution reusable in a real framework? [FOLLOW-UP]

Candidate Answer — the ladder of reuse:

1. **Extract to a page object** — selectors + steps become methods on the page class; specs call intent, not mechanics.
2. **Extract to a custom fixture** (`test.extend`) when setup/teardown is involved — lazy creation, automatic cleanup, injected by name (Section 8).
3. **Parameterize what varies** — selectors via the page object, URLs/env via config (`baseURL`, dotenv), data via JSON/fixtures — never hardcode.
4. **Utilities get types** — a typed `ExcelReader`, `ApiHelper`, `DataFactory` in utils/ with an index.ts barrel export.
5. **At company scale** — publish the framework as an internal npm package that several product suites depend on, with versioning and a changelog; enforce quality with lint rules (e.g., no raw locators in specs, no waitForTimeout).

Q32. What edge cases would you test? [FOLLOW-UP]

Candidate Answer — the checklist to recite (pick per feature):

- **Input edges:** empty input, whitespace-only, very long strings, special characters (é, emoji, quotes), unicode/RTL, null, max boundary values (upload size limits!).
- **State edges:** zero results ("no results found" state), duplicates, expired session mid-flow, going back/refresh mid-wizard.
- **Environment edges:** slow network / offline (`test.use({ offline: true })`), mobile viewport, dark mode, locale/timezone.
- **Concurrency:** double-click submit (double submission), two tabs on the same account, parallel data mutation.
- **Failure paths:** API 500 responses (mock with `route.fulfill`) — error banners, retry buttons.

SAY THIS: "I test the happy path, the boundaries, and the failure paths — for uploads specifically: empty file, oversized file, special characters in the file name, and a server error during upload."

Q33. How would you optimize the solution for very large input? [FOLLOW-UP]

Candidate Answer:

- **Don't verify bulk through the UI** — asserting 100k rows in the browser is slow and brittle: verify the COUNT via API/DB, spot-check a handful of rows in the UI, sample randomly.
- **Paginate** — iterate pages of data, not one giant list.
- **Parallelize the suite** — more workers, sharding across CI machines; isolate data so parallelism is safe (Q27).
- **Trim evidence** — videos/traces on-failure only; headless runs; `reducedMotion`.
- **In helper code** — single-pass O(n) algorithms, generators/streams instead of materializing giant arrays, Set/Map instead of repeated `.find()` scans ($O(n^2)$ -> O(n)).
- **Reuse sessions** — storageState so every test doesn't pay the login cost.

Q34. How would you debug code that works locally but fails in CI? [FOLLOW-UP]

Candidate Answer — the method (order matters):

1. **Reproduce the exact command** — run the identical `npx playwright test` (same flags, same shard/grep) locally first; half the cases end here (a stale local assumption, an unsaved file, a different branch).
2. **Capture evidence from the failure itself** — traces, videos, screenshots configured `on-failure` in CI; the trace shows DOM, network, console per step — read it before changing anything.

3. **Diff the environment** — env vars/secrets (the classic: token present locally, absent in CI), different baseURL/data, timezone/locale, OS differences (case-sensitive file paths on Linux CI vs Windows local!), Node/browser versions.
4. **Isolate** — `--grep` the single failing test, `--workers=1`, `--project=chromium` only; if it passes alone, it's an interaction/parallelism issue (-> Q27 data isolation), if it fails alone, it's environmental (-> Q26 headless/viewport checks).
5. **Parity option** — run the suite in the same Docker image as CI locally; if it fails there too, you can now iterate quickly instead of waiting for CI round-trips.

Interviewer Follow-up: *"What's your general principle?"*

> *"Never debug by guessing — reproduce exactly, read the trace, diff the environment, then bisect. The failure is deterministic somewhere; your job is to find WHERE it's deterministic."*

SECTION 28 — APPENDIX: THE FINAL REFERENCE PAGES (config file, framework design, libraries, commands)

28.1 The COMPLETE playwright.config.ts — everything we covered, in ONE file

```
// playwright.config.ts
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({

  // ===== 1. WHAT & WHERE TO RUN =====
  testDir: './src/tests',              // where the spec files live
  fullyParallel: true,                 // even tests inside ONE file run in parallel
  retries: process.env.CI ? 2 : 0,    // retries ONLY in CI (fail-then-pass = marked flaky)
  forbidOnly: !!process.env.CI,       // blocks accidentally-committed test.only in CI

  // ===== 2. TIME BUDGETS (the timeout family) =====
  timeout: 30_000,                    // TOTAL budget per test (default 30s)
  expect: { timeout: 5_000 },          // budget per web-first assertion (default 5s)

  // ===== 3. DEFAULTS INJECTED INTO EVERY TEST (the "use" block) =====
  use: {
    baseURL: 'https://staging.myapp.com',  // page.goto('/login') just works
    actionTimeout: 15_000,                 // budget per single action (click/fill)
    navigationTimeout: 20_000,             // budget for goto()/reload
    screenshot: 'only-on-failure',          // evidence — captured always, kept on failure
    video: 'retain-on-failure',
    trace: 'retain-on-failure',             // trace.zip for the Trace Viewer
    locale: 'en-IN',
    timezoneId: 'Asia/Kolkata',
  },

  // ===== 4. PROJECTS — the cross-browser + device + role matrix =====
  projects: [
    // (a) the SETUP project — logs in ONCE, saves the session (Q20)
    { name: 'setup', testMatch: /auth\.setup\.ts/ },

    // (b) desktop browsers — depend on setup, inherit the saved session
    {
      name: 'chromium',
      dependencies: ['setup'],               // run setup FIRST
      use: { ...devices['Desktop Chrome'],
             storageState: '.auth/user.json' },  // every test starts LOGGED IN
    },
    {
      name: 'firefox',
      dependencies: ['setup'],
      use: { ...devices['Desktop Firefox'], storageState: '.auth/user.json' },
    },
    {
      name: 'webkit',
      dependencies: ['setup'],
      use: { ...devices['Desktop Safari'], storageState: '.auth/user.json' },
    },

    // (c) role-based project — different storageState file
    {
      name: 'admin',
      dependencies: ['setup'],
      use: { ...devices['Desktop Chrome'], storageState: '.auth/admin.json' },
    },

    // (d) mobile emulation — built-in device catalog
    {
      name: 'mobile',
      use: { ...devices['iPhone 14'] },
    },
  ],

  // ===== 5. REPORTING (multiple reporters at once) =====
  reporter: [
```

```
    ['list'],                                    // console progress
    ['html', { open: 'never' }],                 // playwright-report/index.html
    ['json', { outputFile: 'results.json' }],   // machine-readable, for CI tools
  ],

  // ===== 6. ONCE PER RUN (optional) =====
  // globalSetup: './global-setup.ts',       // seed DB / start a test server
  // globalTeardown: './global-teardown.ts',

  // ===== 7. WHERE ARTIFACTS LAND =====
  outputDir: 'test-results',
});
```

How the pieces connect (recite this): the setup project runs FIRST (guaranteed by `dependencies`), saves `.auth/user.json`, and every browser project's `page` fixture injects those cookies while building each test's isolated context — so `page.goto('/dashboard')` in any spec arrives already logged in. 4 projects x 10 specs = 40 test runs, parallel across workers, no Grid.

Every line is an interview answer: `timeout` -> "30s total per test"; `retries` -> "CI-only flake net, pass-on-retry marked flaky"; `dependencies` -> "setup ordering guarantee"; `storageState` -> "login once per run"; `devices` -> "built-in mobile emulation"; `reporter` -> "HTML + list + json together".

28.2 Our framework in detail — how we designed it, and how we use it

Understand it: this is the answer to "walk me through how you designed your framework and how you use it day to day."

The complete project outline (be able to draw this):

```
playwright-framework/
|- package.json                <- deps + npm scripts (npm run smoke / regression)
|- playwright.config.ts        <- the BRAIN (everything in 28.1)
|- tsconfig.json               <- TypeScript configuration
|- .env                        <- URLs + credentials per environment (dotenv)
|- .gitignore                  <- .auth/, playwright-report/, test-results/
|- global-setup.ts             <- (optional) once-per-run environment work
|
|- src/
|   |- pages/                  <- POM: LoginPage.ts, CartPage.ts, HeaderComponent.ts
|   |- fixtures/               <- test.extend: custom fixtures (pages, data, api client)
|   |- utils/                  <- ExcelReader (exceljs), Logger (winston), helpers
|   |- tests/
|       |- smoke/              <- suite folders -> run via --project=smoke
|       |- regression/
|       |- api/                <- API tests using the request fixture
|       |- auth.setup.ts       <- the storageState login-once project
|
|- test-data/                  <- users.json, orders.xlsx, CSV files
|- .auth/                      <- user.json, admin.json storageState (gitignored)
|- playwright-report/          <- generated HTML report (gitignored)
|- test-results/               <- traces, videos, screenshots (gitignored)
```

The design decisions — say each as a WHY answer:

1. **Page Object Model** — one class per page; locators live with their page, specs stay clean. Locator change = one file.
2. **Custom fixtures instead of POM constructors** — pages and data arrive as injected services (`async ({ loginPage }) =>`), created lazily, cleaned up automatically even on failure. Dependency injection for tests.
3. **storageState setup project** — login ONCE per run, every test starts authenticated but still isolated (Section 12 + Q20).
4. **Suites as folders + projects** — smoke/regression/api are project names with `testMatch`, run by `--project=smoke`, wrapped in npm scripts. No grep needed for the daily work.
5. **Data layer by type** — JSON imported natively, Excel via exceljs, CSV via csv-parse, unique random data via faker, environment data via dotenv. Nothing hardcoded in specs.
6. **Evidence by config, not code** — trace/screenshot/video on failure is a config line; in Selenium we wrote listeners for this, in Playwright we don't write any.

7. **Winston for logs, HTML + Allure for reports** — winston transports (console + file) replace Log4j appenders.
8. **CI** — GitHub Actions/Jenkins runs `npx playwright test`, sharded across machines, CI-only retries, publishes the HTML report and traces as artifacts.

How we use it day to day (the "how we use it" answer): "I write specs in the tests folder calling page objects; data comes from test-data via fixtures; running `npm run smoke` executes the smoke project; when something fails, Jenkins serves the HTML report and I open the trace.zip for the exact failing test — DOM snapshots, network, console, step by step — then fix and push."

MEMORY HOOK: *"Pages hold locators, fixtures inject everything, setup saves the login, config holds the power, folders define the suites."*

28.3 The library stack — "if we use Excel, which library do we use?"

Q: Which library reads Excel test data in a Playwright (TypeScript) framework?

Candidate Answer: exceljs — the Node equivalent of Apache POI (reads AND writes .xlsx; `xlsx`/SheetJS is the lighter alternative for read-only).

```
// utils/ExcelReader.ts — the POI equivalent
import ExcelJS from 'exceljs';

export async function readExcel(sheetName: string, file = 'test-data/data.xlsx') {
  const wb = new ExcelJS.Workbook();
  await wb.xlsx.readFile(file);
  const sheet = wb.getWorksheet(sheetName);

  const rows: Record<string, string>[] = [];
  let headers: string[] = [];
  sheet?.eachRow((row, idx) => {
    const values = (row.values as string[]).slice(1);
    if (idx === 1) { headers = values.map(String); return; }   // row 1 = header
    const obj: Record<string, string> = {};
    values.forEach((v, i) => { obj[headers[i]] = String(v); });
    rows.push(obj);
  });
  return rows;
}

// feeding tests — the DataProvider equivalent:
import { readExcel } from '../utils/ExcelReader';

for (const row of await readExcel('LoginData')) {
  test(`login as ${row.username} expects ${row.expected}`, async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Username').fill(row.username);
    await page.getByLabel('Password').fill(row.password);
    await expect(page.getByRole('alert')).toContainText(row.expected);
  });
}
```

The full stack table (what we use for every need):

Need	What we use	Selenium-world equivalent
Runner + automation	@playwright/test 1.62.x	Selenium + TestNG + Maven
Excel data (.xlsx)	**exceljs** (or xlsx/SheetJS)	Apache POI
JSON data	native `import` / fs	Jackson / JSON-simple
CSV data	csv-parse	OpenCSV
Environment config	dotenv (.env files)	config.properties
Logging	winston (console + file transports)	Log4j2
Reporting	built-in HTML + allure-playwright	Extent Reports / Allure
Random test data	@faker-js/faker	Java Faker
API testing	built-in `request` fixture	RestAssured

Need	What we use	Selenium-world equivalent
Language + runtime	TypeScript on Node 20 LTS	Java
Suites + build	projects + npm scripts + CI	testng.xml + Maven + Jenkins
BDD (optional)	playwright-bdd	Cucumber-Java

MEMORY HOOK: *"Excel = exceljs, env = dotenv, logs = winston, data = faker, reports = HTML/Allure."*

28.4 Commands — from ZERO to running (memorize these cold)

```
# ===== SETUP (once per machine / fresh clone) =====
npm init playwright@latest        # scaffold: config, package.json, example spec, folders
npm i -D @playwright/test         # add the dependency (scaffold does this for you)
npx playwright install            # download the browser binaries (chromium, firefox, webkit)
npx playwright install chromium   # just one browser

# ===== RUNNING =====
npx playwright test               # whole suite
npx playwright test --headed      # watch the browser
npx playwright test --project=smoke       # one suite (project)
npx playwright test tests/smoke/login.spec.ts   # one file
npx playwright test --grep "@smoke"       # tag filter (one-off use)
npx playwright test --workers=4           # control parallelism
npx playwright test --shard=1/3           # split across 3 CI machines
npx playwright test --timeout=60000       # override per-test timeout

# ===== DEBUGGING & TOOLING =====
npx playwright test --debug       # Inspector — step through line by line
npx playwright test --ui          # UI Mode — watch mode + time travel
npx playwright codegen https://site.com   # record actions, prints best-practice code
npx playwright show-trace test-results/.../trace.zip   # open a trace
npx playwright show-report        # open the last HTML report

# ===== MAINTENANCE =====
npm update @playwright/test      # upgrade the runner
npx playwright install           # ...then ALWAYS re-run: browsers are version-locked
```

MEMORY HOOK (the order you'll actually type them in a new job): *"init -> install -> test -> debug."*

This is the end of the guide. If you can explain every section in the INDEX from memory — the framework script, the architecture, the assignments, and these final reference pages — you are ready for the interview. Go get it.